

Министерство образования и науки Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ЯДЕРНЫЙ УНИВЕРСИТЕТ
«МИФИ»
(НИЯУ МИФИ)

ОТЧЕТ
О НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
«Методика обнаружения несанкционированных действий и аномальной
активности в корпоративной сети с использованием нейронных сетей»

Исполнитель:

ст. группы №Б22-505

Д.И. Титов

подпись, дата

Научный руководитель:

Д.П. Ланской

подпись, дата

Зам. зав. кафедры № 42:

К.Г. Когос

подпись, дата

Москва 2026

РЕФЕРАТ

Отчёт о научно-исследовательской работе содержит 70 с., 4.11 рис., 4.11 табл., 4.11 источников, 3 приложения.

Ключевые слова: обнаружение вторжений, NIDS, NTA, анализ сетевого трафика, глубокое обучение, нейронные сети, CNN-LSTM, UNSW-NB15, CICIDS2017, ИИ-злоумышленник, дрейф распределений, обнаружение аномалий, фокальная функция потерь, температурное масштабирование, PSI, MMD.

Объект исследования — сетевой трафик корпоративной сети, подверженной несанкционированным действиям и аномальной активности.

Предмет исследования — методика обнаружения аномальной активности и несанкционированных действий с использованием нейронных сетей и активного тестирования посредством ИИ-злоумышленника.

Цель работы — разработка методики обнаружения аномальной активности и несанкционированных действий в корпоративной сети на основе нейронных сетей, включающей анализ сетевого трафика, классификацию угроз и моделирование атак с помощью ИИ-злоумышленника, генерирующего трафик в реальном времени для тестирования системы.

Методы исследования. Систематический анализ литературы по NIDS и глубокому обучению; математическое моделирование атак параметризованными шаблонами с конечным автоматом планирования и drift-инжектором; обучение нейронных сетей и классических ML-моделей в равных условиях с фиксированной схемой препроцессинга; статистический анализ результатов через bootstrap-доверительные интервалы 95 % и парный критерий Уилкоксона.

Основные результаты. Сформирована методика обнаружения, включающая препроцессинг flow-данных, библиотеку из семи параметризованных шаблонов атак, конечный автомат планирования режимов *норма/атака/дрейф* (11 состояний) и блок drift-monitoring на основе индексов PSI, KL и MMD. Реализован программный прототип на стеке Python/PyTorch с интерфейсами FastAPI и Streamlit (более 4200 строк кода, 9 модулей юнит-тестов). На датасете UNSW-NB15 проведено шесть экспериментов (E1–E6) с 22 запусками и 14 моделями. Лучшее F1 (среднее по трём seed) — 0,919 у XGBoost; среди DL-моделей

лидируют CNN-LSTM, TCN и BiLSTM с F1 в диапазоне 0,77–0,84 при FPR порядка 0,01–0,03 после калибровки порога под целевую частоту ложных срабатываний. Inference-задержка CNN-LSTM на CPU составила 0,96 мс/окно, что удовлетворяет требованиям эксплуатации в составе SOC. Гипотеза исследования подтверждена в части роли ИИ-злоумышленника и уточнена по части превосходства DL над классическими baselines на табличных flow-данных UNSW-NB15.

Степень внедрения. Программный прототип допускает развёртывание в составе подсистемы NTA-аналитики корпоративного SOC. В отчёте сформированы инженерные рекомендации по интеграции и калибровке порогов под целевой эксплуатационный FPR.

Область применения. Системы обнаружения вторжений корпоративного класса; учебно-исследовательские стенды по сетевой безопасности; задачи аудита устойчивости NIDS к дрейфу и вариативности атак.

Содержание

РЕФЕРАТ	2
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ	7
ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ	9
ВВЕДЕНИЕ	11
1 Теоретический анализ методов обнаружения аномалий и обоснование требований к системе	14
1.1 Корпоративная сеть как объект мониторинга	14
1.2 Классификация и принципы построения IDS	15
1.3 Методы обнаружения аномалий	15
1.4 Глубинное обучение для анализа flow-данных	16
1.5 Моделирование атак с использованием ИИ	17
1.6 Сравнительный анализ датасетов NIDS	18
1.7 Метрики качества	19
1.8 Формализация требований и гипотеза	20
2 Сравнительный анализ нейросетевых архитектур и проектирование методики обнаружения	23
2.1 Принципы сравнения и методологические замечания	23
2.2 Опубликованные результаты применения архитектур в NIDS .	23
2.3 Обоснование выбора CNN-LSTM как основной архитектуры .	25
2.4 Теоретическая модель данных и pipeline обработки	25
2.4.1 Схема признаков UNSW-NB15	25
2.4.2 Pipeline обработки	26
2.4.3 Унификация признаков с CICIDS2017 для cross-evaluation	27
2.4.4 Управление дисбалансом классов	29
2.5 Архитектура ИИ-злоумышленника	29
2.6 Протокол экспериментов	29

3	Программная реализация прототипа системы обнаружения и ИИ-злоумышленника	32
3.1	Стек и общая структура программного проекта	32
3.2	Подсистема обработки данных	34
3.2.1	Схема и загрузка	34
3.2.2	Препроцессор	34
3.2.3	Скользящие окна	34
3.3	Реализованные модели	34
3.4	Подсистема обучения и оценки	35
3.5	ИИ-злоумышленник	38
3.5.1	Шаблоны атак	38
3.5.2	Конечно-автоматный планировщик	39
3.5.3	Drift-инжектор и runtime	40
3.6	Подсистема online-инференса и интерфейс	40
3.6.1	FastAPI-сервис	40
3.6.2	Формирование алертов	41
3.6.3	Streamlit-дэшборд	41
3.7	Воспроизводимость и журналирование	41
3.8	Покрывтие тестами	41
3.9	Соответствие реализации функциональным требованиям . . .	42
4	Экспериментальное исследование разработанной методики и обоснование выбора модели	44
4.1	Условия экспериментов	44
4.1.1	Параметры разделения и препроцессинга	44
4.2	Эксперимент E1: сравнение моделей-кандидатов	45
4.2.1	Цель и условия	45
4.2.2	Результаты	45
4.2.3	Анализ E1	46
4.3	Эксперимент E2: анализ ошибок CNN-LSTM	47
4.4	Эксперимент E3: калибровка порогов	49
4.5	Эксперимент E4: стресс-тест с ИИ-злоумышленником	50
4.5.1	Цель и условия	50
4.5.2	Результаты	50
4.6	Эксперимент E5: устойчивость к дрейфу распределений . . .	52

4.6.1	Цель и условия	52
4.6.2	Результаты	52
4.7	Эксперимент Е6: эксплуатационная производительность . . .	53
4.8	Демонстрационный прогон	54
4.9	Кривые обучения DL-моделей	55
4.10	Обоснование выбора CNN-LSTM для построения системы . .	56
4.11	Подтверждение и уточнение гипотез исследования	57
ЗАКЛЮЧЕНИЕ		60
ПРИЛОЖЕНИЯ		68
	Приложение А. Полная таблица результатов Е1	68
	Приложение Б. Структура программного прототипа	69
	Приложение В. Конфигурации экспериментов	69

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

Несанкционированные действия в сети

действия, нарушающие установленную политику безопасности корпоративной информационной системы, в том числе попытки получения неавторизованного доступа, эксфильтрации данных, исчерпания ресурсов, скрытого латерального перемещения и развёртывания вредоносного кода.

Аномальная сетевая активность

поведение сетевых соединений, статистически отклоняющееся от характерного для контролируемой инфраструктуры профиля нормы, проявляющееся в признаках длительности, направления, объёма и последовательности соединений.

Сетевой поток (flow)

агрегированная единица сетевого трафика, описываемая совокупностью характеристик (*src/dst IP, src/dst port, protocol*) в пределах заданного временного интервала и характеризуемая длительностью, числом переданных пакетов и байтов, межпакетными интервалами и сопутствующими статистиками.

NIDS (Network-based Intrusion Detection System)

сетевая система обнаружения вторжений, осуществляющая анализ проходящего трафика для выявления попыток несанкционированного доступа.

NTA (Network Traffic Analysis)

подкласс систем мониторинга сетевой безопасности, ориентированный на анализ метаданных сетевых соединений (*flow-level*) с применением статистических и машинно-обучаемых моделей.

Аномальный детектор

программный модуль, формирующий непрерывный скоринг $s(x): X \rightarrow \mathbb{R}$ и принимающий решение об аномальности наблюдения x при превышении порога τ .

Скользящее окно (sliding window)

последовательность из W подряд идущих flow-записей, используемая в качестве входа модели; параметры окна — размер W и шаг S .

Дрейф распределений (concept/covariate drift)

изменение совместного распределения $p(x, y)$ или его компонент во времени в продуктивной эксплуатации; источник деградации аномального детектора.

Калибровка вероятностей

процедура согласования выходной вероятности классификатора с эмпирической частотой положительного класса; в работе используется температурное масштабирование.

ИИ-злоумышленник (AI-attacker)

активный программный компонент, формирующий поток сетевых событий с управляемым переключением режимов *норма/атака/дрейф* в рамках программного стенда; в данной работе реализован на основе библиотеки параметризованных шаблонов, конечного автомата планирования и инжектора дрейфа.

SOC (Security Operations Center)

функциональная организационно-техническая структура мониторинга и реагирования на инциденты информационной безопасности корпоративного класса.

Алерт

сформированное системой обнаружения событие безопасности, сопровождаемое уровнем критичности, скорингом, типом предположительной атаки и контекстом наблюдения, направляемое в SOC для последующей обработки аналитиком.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И УСЛОВНЫХ ОБОЗНАЧЕНИЙ

AE	Autoencoder, автоэнкодер
ATT&CK	Adversarial Tactics, Techniques and Common Knowledge — матрица MITRE
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network, свёрточная нейронная сеть
cGAN	Conditional Generative Adversarial Network
CI	Confidence Interval, доверительный интервал
DL	Deep Learning, глубокое обучение
DPI	Deep Packet Inspection, глубокий анализ пакета
ECE	Expected Calibration Error
EDA	Exploratory Data Analysis
EPS	Events Per Second, событий в секунду
FAR	False Alarm Rate
FN	False Negative, ложноотрицательный
FP	False Positive, ложноположительный
FPR	False Positive Rate
FSM	Finite-State Machine, конечный автомат
GAN	Generative Adversarial Network
GRU	Gated Recurrent Unit
IAT	Inter-Arrival Time, межпакетный интервал
IDS	Intrusion Detection System
IPFIX	IP Flow Information Export
IPS	Intrusion Prevention System
IQR	Interquartile Range, межквартильный размах
KDD	Knowledge Discovery in Databases
KL	Kullback–Leibler divergence
KS-test	Kolmogorov–Smirnov test
LSTM	Long Short-Term Memory
MCC	Matthews Correlation Coefficient
ML	Machine Learning, машинное обучение
MLP	Multilayer Perceptron, многослойный перцептрон
MMD	Maximum Mean Discrepancy
NIDS	Network-based Intrusion Detection System

NTA Network Traffic Analysis
OCSVM One-Class Support Vector Machine
OOD Out-Of-Distribution
PR-AUC Area Under the Precision–Recall Curve
PSI Population Stability Index
ReLU Rectified Linear Unit
RF Random Forest, случайный лес
RNN Recurrent Neural Network, рекуррентная нейронная сеть
ROC-AUC Area Under the Receiver Operating Characteristic Curve
SOAR Security Orchestration, Automation and Response
SOC Security Operations Center, центр мониторинга безопасности
SVM Support Vector Machine
TCN Temporal Convolutional Network
TN True Negative, истинноотрицательный
TP True Positive, истинноположительный
TPR True Positive Rate, чувствительность (Recall)
TTD Time-To-Detect, время до обнаружения
VAE Variational Autoencoder, вариационный автоэнкодер
XGBoost eXtreme Gradient Boosting
ВКР выпускная квалификационная работа
НИР научно-исследовательская работа

ВВЕДЕНИЕ

Актуальность работы. Корпоративные сети остаются устойчивой целью несанкционированных воздействий, включающих эксфильтрацию данных, латеральное перемещение, разведку, отказ в обслуживании и эксплуатацию уязвимостей сетевых сервисов. Доля зашифрованного трафика в современных корпоративных инфраструктурах превышает 90 %, что делает сигнатурный анализ полезной нагрузки малоэффективным. Центр тяжести разработки систем обнаружения вторжений (*Intrusion Detection Systems, IDS*) переходит в сторону аномального анализа метаданных потоков с применением методов машинного и глубокого обучения. При этом существенной проблемой остаётся воспроизводимая, формализованная оценка таких систем в условиях, приближенных к реальной эксплуатации, в том числе при дрейфе распределений и вариативности атакующих сценариев.

Объект исследования — сетевой трафик корпоративной сети, подверженной несанкционированным действиям и аномальной активности.

Предмет исследования — методика обнаружения аномальной активности и несанкционированных действий с использованием нейронных сетей, а также методология её активного тестирования с помощью программного агента ИИ-злоумышленника.

Цель работы. Разработка методики обнаружения аномальной активности и несанкционированных действий в корпоративной сети на основе нейронных сетей, включающей анализ сетевого трафика, классификацию угроз и моделирование атак с помощью ИИ-злоумышленника, генерирующего трафик в реальном времени для тестирования системы.

Задачи исследования:

1. Изучение и систематизация существующих методов и технологий обнаружения аномалий и атак: статистические подходы, классические ML-методы, нейросетевые модели; обзор IDS/IPS- и NIDS-решений.
2. Изучение методов моделирования атак с использованием ИИ и подходов к генерации синтетического трафика; определение роли ИИ-злоумышленника как агента переключения режимов *норма/атака/дрейф*.
3. Формулировка гипотезы исследования и формализация функциональных и нефункциональных требований к системе.
4. Разработка и обоснование архитектуры нейронной сети, пригодной для

последовательностей и окон сетевого трафика; определение входного представления и схемы алертирования.

5. Сбор и подготовка данных сетевого трафика (*UNSW-NB15, CICIDS2017*), включая препроцессинг, нормализацию, формирование скользящих окон и управление дисбалансом классов.
6. Обучение нейросетевых моделей, настройка гиперпараметров, калибровка порогов под целевой эксплуатационный уровень ложных срабатываний, сравнение с baseline-моделями.
7. Реализация ИИ-злоумышленника и комплексное тестирование прототипа в связке *агент* $\rightarrow$ *детектор*.
8. Реализация программного прототипа мониторинга и детекции в реальном времени с подсистемами интеграции с источниками сетевой телеметрии.
9. Оценка эффективности, оптимизация и доработка прототипа, формирование выводов и направлений дальнейшего развития.

Гипотеза исследования. Использование нейронных сетей для анализа сетевого трафика позволяет с высокой точностью выявлять несанкционированные действия и аномальную активность, проявляющиеся в характерных сетевых паттернах (изменения интенсивности, нетипичные направления, аномальные сессии и последовательности соединений). Применение ИИ-злоумышленника в форме программного агента, генерирующего трафик и атакующие сценарии в реальном времени (включая сценарии с дрейфом распределений), обеспечивает полноту и воспроизводимость тестирования, повышает устойчивость детектора и снижает нагрузку на SOC за счёт уменьшения ложных срабатываний и более точной приоритизации инцидентов.

Методы исследования. Систематический анализ литературы; математическое моделирование (вероятностные и информационно-теоретические меры дрейфа, формальные определения функций потерь и метрик); обучение нейронных сетей и классических моделей в равных условиях; статистический анализ результатов через bootstrap-доверительные интервалы и парный критерий Уилкоксона.

Научная новизна. Новизна состоит в комбинации трёх компонент: (а) гибридной архитектуры CNN-LSTM для классификации flow-окон, обоснованной теоретическим и экспериментальным сравнением с десятью альтернативами в равных условиях; (б) ИИ-злоумышленника как воспроизводи-

мого активного источника тестовых сценариев на основе шаблонов, конечного автомата и drift-инжектора, без использования генеративных моделей; (в) единого протокола экспериментов, включающего сравнение моделей в равных условиях, анализ ошибок, калибровку порога под эксплуатационный FPR, стресс-тестирование с агентом, drift-устойчивость и эксплуатационную производительность.

Практическая значимость. Разработанный прототип реализован на стеке Python / PyTorch с REST-интерфейсом на FastAPI и визуальной панелью на Streamlit и может быть интегрирован в подсистему сетевой аналитики корпоративного SOC. Зафиксированы рекомендации по интеграции с источниками NetFlow / IPFIX, калибровке порогов под допустимый FPR и оценке устойчивости к дрейфу.

Структура отчёта. Основная часть отчёта состоит из четырёх разделов, соответствующих четырём этапам, заявленным в постановке задачи. **Этап 1** содержит теоретический анализ методов обнаружения аномалий, обзор архитектур нейронных сетей, сравнение датасетов NIDS, обзор методов моделирования атак с применением ИИ, формализацию требований и гипотезы. **Этап 2** — сравнительный анализ архитектур нейронных сетей по опубликованным NIDS-результатам, обоснование выбора целевой архитектуры, теоретическая модель данных и pipeline, библиотека синтетических тестовых сценариев и протокол экспериментов. **Этап 3** — программная реализация прототипа системы обнаружения аномалий, обучение и оптимизация моделей, разработка ИИ-злоумышленника. **Этап 4** — комплексное тестирование прототипа, анализ эффективности, оптимизация и доработка, оформление итоговых материалов; именно этому этапу уделено наибольшее внимание (расчёт метрик, анализ ошибок, тесты с ИИ-злоумышленником, оценка устойчивости к дрейфу и производительности).

1 Теоретический анализ методов обнаружения аномалий и обоснование требований к системе

Содержание главы соответствует первому этапу постановки задачи (см. ВКР, Этап 1): изучение литературы и существующих методов обнаружения аномалий в сетевом трафике; анализ систем IDS/IPS и их компонентов; обзор методов создания ИИ-злоумышленника; формулировка гипотезы исследования и требований к системе.

1.1 Корпоративная сеть как объект мониторинга

Под корпоративной сетью в настоящей работе понимается совокупность взаимосвязанных программно-аппаратных средств, обеспечивающих обмен данными между рабочими станциями, серверами, сетевым оборудованием и внешними сегментами. С точки зрения мониторинга безопасности корпоративная сеть характеризуется тремя свойствами, существенными для построения NIDS:

1. высокой неоднородностью трафика, приводящей к мультимодальному распределению признаков и затрудняющей применение единых пороговых правил [1; 2];
2. доминированием легитимного трафика, что создаёт сильный дисбаланс классов и требует специализированных методов обучения и оценки [3—5];
3. динамичностью характеристик, приводящей к явлению дрейфа распределений [6; 7].

Объектами мониторинга в работе принимаются: сетевые соединения (flow) на уровне L3/L4, сессии прикладного уровня и временные последовательности соединений в скользящих окнах. Это согласуется с подходом, принятым в большинстве современных систем сетевого анализа трафика и в открытых датасетах [8; 9].

В качестве рабочей таксономии угроз использована классификация датасета UNSW-NB15, выделяющая девять семейств атакующего трафика: *Fuzzers, Analysis, Backdoors, DoS, Exploits, Generic, Reconnaissance, Shellcode, Worms* [8].

1.2 Классификация и принципы построения IDS

В соответствии с устоявшейся в литературе типологией IDS делятся на три класса [1; 2; 10]: **сигнатурные** (совпадение с заранее заданным шаблоном), **аномальные** (отклонение от модели нормы) и **гибридные**. Сигнатурный класс обеспечивает высокую точность по известным угрозам, но не способен обнаруживать ранее не встречавшиеся атаки (*zero-day*). Аномальный класс расширяет покрытие на неизвестные сценарии за счёт повышенной частоты ложных срабатываний и чувствительности к дрейфу.

С точки зрения исходного представления данных различают *packet-based* (анализ полезной нагрузки, DPI) и *flow-based* анализ. Стандартами экспорта flow-записей являются Cisco NetFlow [11] и IETF IPFIX [12]. В настоящей работе принят flow-based подход по причине его совместимости с шифрованным трафиком, ресурсной эффективности и прямой переносимости на инфраструктуры реальных корпоративных сетей.

1.3 Методы обнаружения аномалий

Постановка задачи: для пространства признаков $X \subseteq \mathbb{R}^d$ требуется построить функцию скоринга $s: X \rightarrow \mathbb{R}$ и порог τ такие, что

$$\hat{y}(x) = \mathbb{I}[s(x) > \tau], \quad x \in X, \quad (1.1)$$

где $\hat{y}(x) = 1$ интерпретируется как “аномалия / атака”.

Статистические и энтропийные методы. Простейший подход — z -оценка и многомерное расстояние Махаланобиса:

$$d_M(x) = \sqrt{(x - \mu)^\top \Sigma^{-1} (x - \mu)}. \quad (1.2)$$

Энтропия Шеннона $H(P) = -\sum_a P(a) \log P(a)$ используется для оценки распределений IP-адресов и портов в скользящем окне.

Isolation Forest [13] определяет скоринг через ожидаемую глубину изоляции точки:

$$s(x, n) = 2^{-\frac{\mathbb{E}[h(x)]}{c(n)}}, \quad c(n) \approx 2(\ln(n-1) + 0.5772) - \frac{2(n-1)}{n}. \quad (1.3)$$

Метод имеет линейную сложность и хорошо подходит для unsupervised-

режима.

One-Class SVM [14] ищет в спрямляющем пространстве минимальную гиперсферу, содержащую долю $1 - \nu$ обучающих наблюдений; параметр $\nu \in (0, 1]$ задаёт верхнюю оценку доли выбросов.

1.4 Глубинное обучение для анализа flow-данных

В работе рассмотрено восемь типов нейросетевых архитектур.

MLP реализуется рекурсией $h^{(l)} = \phi(W^{(l)}h^{(l-1)} + b^{(l)})$ и служит нижней границей DL-моделей.

1D-CNN применяет операцию свёртки $(h * w)_t = \sum_{k=0}^{K-1} w_k h_{t-k}$ для извлечения локальных паттернов в скользящем окне.

LSTM [15] вводит ячейку состояния c_t и три гейта:

$$\begin{aligned} i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), \\ f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), \\ \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \\ h_t &= o_t \odot \tanh(c_t). \end{aligned} \tag{1.4}$$

Гейтовая структура обеспечивает устойчивое распространение градиента и способность хранить долгосрочный контекст — свойство, востребованное при обнаружении медленных атак.

GRU [16] — упрощённая RNN с двумя гейтами; **BiLSTM** использует прямой и обратный LSTM для двунаправленного контекста.

Гибрид CNN-LSTM применяет последовательное преобразование

$$x_t \xrightarrow{\text{Conv1D}} z_t \xrightarrow{\text{LSTM}} h_t \xrightarrow{\text{FC}} \hat{p}_t,$$

сочетая локальные паттерны и временные зависимости.

TCN [17] использует причинные дилатированные свёртки с экспоненциально растущим рецептивным полем.

Transformer-encoder [18] основан на само-внимании:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V. \quad (1.5)$$

Autoencoder и VAE. Автоэнкодер минимизирует ошибку реконструкции $\|x - D(E(x))\|^2$; вариационный автоэнкодер [19] обучает вероятностное кодирование максимизацией ELBO:

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x) \| p(z)). \quad (1.6)$$

Качественное сравнение архитектур по характеристикам приведено в таблица 1.1.

Таблица 1.1 – Качественное сравнение нейросетевых архитектур для NIDS

Модель	Локальные паттерны	Временные зависимости	Сложность обучения	Inference latency
MLP	нет	нет	низкая	низкая
1D-CNN	да	ограничено	средняя	низкая
LSTM	нет	да	средняя	средняя
GRU	нет	да	средняя	низкая
BiLSTM	нет	двунаправ.	высокая	средняя
CNN-LSTM	да	да	высокая	средняя
TCN	да	фиксиров.	средняя	низкая
Transformer	через attention	да	высокая	средняя
AE / VAE	косвенно	нет	средняя	низкая

1.5 Моделирование атак с использованием ИИ

В литературе выделяют три семейства подходов к формированию атакующего трафика для тестирования NIDS:

1. **Шаблонная (*rule-based, template-based*) генерация:** атаки задаются параметрическими шаблонами потоков; применение — от классических работ DARPA/KDD до утилит *traffic replay* [20; 21].

2. **Генеративные модели:** GAN, cGAN, WGAN, VAE для синтеза реалистичных flow-векторов и последовательностей. Эталонная работа — Ring и соавт. [22]; вариационные подходы — [23].
3. **Adversarial-агенты на обучении с подкреплением:** агент обучается формировать последовательности действий, обходящих детектор [24].

Классическая постановка GAN [25]:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]. \quad (1.7)$$

Conditional GAN [26] вводит обуславливающую переменную c (тип атаки); WGAN-GP [27; 28] заменяет JS-расстояние Wasserstein-1 и обеспечивает 1-липшицевость дискриминатора через градиентный штраф.

В рамках настоящей работы обоснован выбор **гибридной шаблонной архитектуры** ИИ-злоумышленника:

Templates + FSM-планировщик + Drift-инжектор

без cGAN-составляющей. Аргументы: высокая воспроизводимость и прозрачная ground-truth, семантическая согласованность шаблонов с таксономией UNSW-NB15, отсутствие риска коллапса режима, интерпретируемость для аналитика SOC. cGAN сохраняется в обзоре как теоретическая альтернатива и направление дальнейшего развития.

1.6 Сравнительный анализ датасетов NIDS

В таблица 1.2 приведена сводная экспертная оценка десяти публично доступных датасетов NIDS по восьми критериям, систематизированным в [9; 21].

Таблица 1.2 – Сводное сравнение публично доступных датасетов NIDS

Датасет	K1	K2	K3	K4	K5	K6	K7	K8: ограничения
KDDCup99 [29]	–	–	–	○	○	–	+	устаревшая модель угроз
NSL-KDD [29]	–	–	○	+	○	–	+	таксономия 1998 г.; малый объём для DL
UNSW-NB15 [8]	+	+	+	○	+	+	+	доминирование Generic
CICIDS2017 [9]	+	+	○	○	+	+	+	дефекты разметки; дубликаты
CSE-CIC-IDS2018 [9]	+	+	○	○	+	+	○	унаследованные дефекты
TON_IoT [30]	+	○	+	○	+	+	+	ориентация на IoT
Bot-IoT [31]	○	–	+	–	+	+	+	доля атак > 97%
Kyoto 2006+ [32]	○	○	○	○	○	○	○	honeypot-смещение
DARPA [20]	–	–	○	○	–	○	+	1998 г.; искусственный трафик
LITNET-2020 [33]	+	+	○	○	+	+	○	таксономический дисбаланс

K1 — актуальность; K2 — репрезентативность нормы; K3 — качество разметки; K4 — дисбаланс классов; K5 — пригодность для DL; K6 — flow-формат; K7 — воспроизводимость; K8 — ограничения. “+” — соответствует, “○” — частично, “–” — нет.

В работе принят датасет **UNSW-NB15** как основной (наибольшее покрытие критериев), **CICIDS2017** — как вспомогательный для кросс-валидации.

1.7 Метрики качества

Для бинарной задачи введены матрица ошибок (TP, TN, FP, FN) и производные метрики:

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (1.8)$$

$$F1 = \frac{2 \text{Precision Recall}}{\text{Precision} + \text{Recall}}, \quad \text{FPR} = \frac{FP}{FP + TN}. \quad (1.9)$$

MCC (Matthews correlation coefficient) рекомендуется при сильном дисбалансе:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (1.10)$$

ROC-AUC и **PR-AUC** интегрируют качество модели по всем порогам; PR-AUC более чувствителен к редкому классу и принят как основная интегральная метрика. К эксплуатационным метрикам относятся *time-to-detect* (TTD), *inference latency*, *throughput* и *FP-per-time*.

Температурное масштабирование [34] согласует выходную вероятность с эмпирической частотой:

$$\hat{p}_T(y = 1 \mid x) = \sigma\left(\frac{z(x)}{T}\right), \quad (1.11)$$

где $z(x)$ — логит, T — параметр, подбираемый на валидационной выборке. Качество калибровки оценивается по ECE (*Expected Calibration Error*).

Метрики дрейфа. PSI (Population Stability Index), KL-расхождение и MMD (Maximum Mean Discrepancy):

$$\text{PSI} = \sum_b (p_b - q_b) \ln \frac{p_b}{q_b}, \quad (1.12)$$

$$\text{MMD}^2(P, Q) = \mathbb{E}_{P,P}[k(x, x')] - 2\mathbb{E}_{P,Q}[k(x, y)] + \mathbb{E}_{Q,Q}[k(y, y')]. \quad (1.13)$$

1.8 Формализация требований и гипотеза

Концептуальная модель системы состоит из трёх подсистем: предобработки, детекции и активного тестирования (ИИ-злоумышленник). Сформированы 10 функциональных и 9 нефункциональных требований; ключевые количественные пороги приведены в таблица 1.3.

Таблица 1.3 – Ключевые нефункциональные требования к системе

ID	Содержание	Порог
NFR-1	F1 на UNSW-NB15 (binary)	$\geq 0,90$
NFR-1	PR-AUC на UNSW-NB15	$\geq 0,95$
NFR-2	FPR при Recall $\geq 0,85$	$\leq 0,01$
NFR-3	Inference latency (CPU, batch=1)	≤ 50 мс/окно
NFR-4	Throughput (CPU, батч)	≥ 1000 EPS
NFR-5	Time-to-detect (DDoS / Scan / Brute)	≤ 1 окна
NFR-6	$\Delta F1$ при PSI $\leq 0,25$	$\leq 0,10$
NFR-7	Перенос UNSW $\rightarrow$ CICIDS2017 без дообуч. ($\Delta F1$)	$\leq 0,20$
NFR-8	Воспроизводимость одной командой	да
NFR-9	Стабильность σ_{F1} ($n \geq 5$ seed)	$\leq 0,01$

Формально сформулированы три гипотезы исследования: основная H_0^{main} о превосходстве CNN-LSTM над ансамблем baselines на UNSW-NB15; дополнительная H_0^{aux} о роли ИИ-злоумышленника в воспроизводимом тестировании; гипотеза переносимости H_0^{transfer} на CICIDS2017. Проверка проводится на этапе 4.

Выводы по главе 1

1. Корпоративная сеть как объект мониторинга характеризуется неоднородностью трафика, дисбалансом классов и динамичностью распределений; обоснован выбор flow-уровня анализа, совместимого с NetFlow/IPFIX.
2. Сигнатурные, аномальные и гибридные подходы NIDS дополняют, а не заменяют друг друга; в работе принят аномальный нейросетевой подход.
3. Рассмотрены восемь типов нейросетевых архитектур, для каждой приведены математические основы; гибрид CNN-LSTM покрывает оба измерения структуры flow-данных и принимается в качестве основной proposed-архитектуры.
4. Систематизированы три семейства подходов к моделированию атак; обоснован выбор гибридной шаблонной архитектуры ИИ-злоумышленника

без cGAN-составляющей.

5. Из десяти датасетов выбран UNSW-NB15 как основной и CICIDS2017 как вспомогательный.
6. Сформированы количественные требования и гипотезы исследования.

2 Сравнительный анализ нейросетевых архитектур и проектирование методики обнаружения

Содержание главы соответствует второму этапу постановки задачи (см. ВКР, Этап 2): анализ и сравнение архитектур нейронных сетей; формирование теоретической модели и требований к данным; генерация и обоснование синтетических тестовых сценариев.

2.1 Принципы сравнения и методологические замечания

Цель главы — обоснование выбора нейросетевой архитектуры и формализация методики обработки данных и тестирования. Сравнение архитектур проводится в два этапа: (а) теоретический анализ опубликованных в литературе NIDS-результатов с явной привязкой к первоисточникам (см. таблица 2.1); (б) собственный эксперимент в равных условиях, описываемый на этапе 4.

Методологическое замечание. Прямое количественное сравнение результатов из разных литературных источников некорректно из-за: (а) различий датасетов (NSL-KDD, KDDCup99, UNSW-NB15, CICIDS2017); (б) различий схем препроцессинга и разделения выборок; (в) различной интерпретации метрик при дисбалансе классов [1; 3; 35]. Поэтому литературные результаты используются как качественные индикаторы.

2.2 Опубликованные результаты применения архитектур в NIDS

В таблица 2.1 сведены опубликованные в рецензируемых источниках NIDS-результаты с указанием датасета и постановки.

Таблица 2.1 – Опубликованные результаты применения нейронных сетей в задачах NIDS

Источник	Архитектура	Датасет	Задача	Результат
Kim и соавт. [36]	LSTM-RNN	KDD Cup 99	binary	Accuracy $\approx$ 96,93%; DR $\approx$ 98,88%; FAR $\approx$ 10,04%
Yin и соавт. [37]	RNN	NSL-KDD (KDDTest+)	binary	Accuracy $\approx$ 83,28% при 80 hidden units

Источник	Архитектура	Датасет	Задача	Результат
Yin и соавт. [37]	RNN	NSL-KDD (KDDTest+)	multiclass	Accuracy $\approx$ 81,29%
Moustafa Slay [38]	и ANN	UNSW-NB15	binary	Accuracy $\approx$ 81,3%; FAR $\approx$ 18,4%
Vinayakumar соавт. [39]	и DNN, CNN, LSTM, CNN- LSTM	NSL-KDD, KDDCup99, UNSW-NB15, CICIDS2017	binary, multiclass	CNN-LSTM показал наи- большие F1/Аккурату сре- ди DL-семейства данной работы на UNSW-NB15 и CICIDS2017
Mirsky соавт. [40]	и Kitsune (ансамбль AE)	собственный стенд (Mirai, supervised, SSDP Flood и др.)	semi- online	AUC: Mirai $\approx$ 0,9988; SSDP $\approx$ 0,9844; ARP MitM $\approx$ 0,9606
Sharafaldin и со- авт. [9]	MLP, KNN	RF, CICIDS2017	multiclass	MLP: Precision $\approx$ 0,77, Recall $\approx$ 0,83; RF/KNN превосходят MLP в той же работе
Ahmad соавт. [2] (обзор)	и LSTM, BiLSTM, CNN-LSTM	NSL-KDD, UNSW-NB15, CICIDS2017	binary, multiclass	BiLSTM и CNN-LSTM устойчиво превосходят LSTM по Accuracy/F1

Выявленные тенденции в литературе:

1. Гибридные модели (CNN-LSTM, BiLSTM) устойчиво превосходят как чисто свёрточные, так и чисто рекуррентные модели по F1 в *внутренних* сравнениях соответствующих работ.
2. Высокие показатели на KDD-семействе ($> 95\%$ Аккурату) не должны интерпретироваться как доказательство практической пригодности из-за известных дефектов датасета [3; 29].
3. На современных датасетах (UNSW-NB15, CICIDS2017) разрыв между классическими ML- и DL-моделями существенно меньше [2; 10].

2.3 Обоснование выбора CNN-LSTM как основной архитектуры

С учётом теоретического анализа в качестве основной (*proposed*) архитектуры выбирается **CNN-LSTM**. Решение обосновано следующими аргументами.

1. **Соответствие природе данных.** Атаки в flow-представлении проявляются одновременно как локальные паттерны (например, специфическое сочетание признаков SYN-флуда в одном flow) и как временные тренды (последовательность подобных flow). CNN-LSTM моделирует обе компоненты в одной архитектуре.
2. **Опубликованные результаты.** В нескольких независимых рецензируемых работах [2; 10; 39; 41] CNN-LSTM демонстрирует наибольшие F1/Ассурасу в внутренних сравнениях на UNSW-NB15 и CICIDS2017.
3. **Воспроизводимость.** Архитектура хорошо документирована, реализуется стандартными примитивами PyTorch, не требует специализированных техник обучения.
4. **Эксплуатационная стоимость.** Inference latency укладывается в диапазон, приемлемый для NTA-pipeline.
5. **Альтернативы и контрольные модели.** В качестве альтернатив в собственном эксперименте сохраняются BiLSTM (двунаправленный контекст) и Transformer-encoder (attention); в качестве контрольных semi-supervised baselines — AE и VAE; в качестве классических baseline — Logistic Regression, Random Forest, XGBoost, Isolation Forest, One-Class SVM.

2.4 Теоретическая модель данных и pipeline обработки

2.4.1 Схема признаков UNSW-NB15

UNSW-NB15 (*cleaned* распределение) содержит 44 содержательных признака, разделяемых на пять групп (таблица 2.2).

Таблица 2.2 – Группы признаков UNSW-NB15 и их назначение

Группа	Состав	Роль в детекции
Базовые потоковые	<i>dur, proto, service, state, sbytes, dbytes, sttl, dttl, sloss, dloss, rate</i>	длительность, протокол, состояние сессии, байты и пакеты в обе стороны, общая скорость потока
Контентные	<i>sload, dload, swin, dwin, smean, dmean, stcpb, dtcpb, trans_depth, response_body_len, spkts, dpkts</i>	скорости, окна TCP, базовые транзакционные характеристики
Временные	<i>sjit, djit, sinpkt, dinpkt, tcprtt, synack, ackdat</i>	межпакетные интервалы, джиттеры, RTT, timing-сигнатуры
Дополнительные	<i>ct_state_ttl, ct_flw_http_mthd, is_ftp_login, ct_ftp_cmd, is_sm_ips_ports</i>	агрегаты по протокольному поведению, бинарные флаги
Контекстные <i>ct_*</i>	<i>ct_srv_src, ct_srv_dst, ct_dst_ltm, ct_src_ltm, ct_src_dport_ltm, ct_dst_sport_ltm, ct_dst_src_ltm</i>	агрегаты по последним 100 соединениям; отражают режим сканирования и фан-аут

2.4.2 Pipeline обработки

Полный pipeline принимает на вход поток flow-записей:

IPFIX/CSV → Schema → Cleanup → Encoding
 → Scaling → Windowing → Model
 → Threshold → Alert

Очистка. Применяются: замена нечисловых пропусков константой, числовых — нулём; унификация регистра и пробелов в *service*, *state*, *proto*; удаление дубликатов; обрезка крайних 0,1% хвостов распределений. Вся статистика собирается *только* на обучающей выборке.

Кодирование категориальных признаков. One-hot для признаков с малой кардинальностью (*proto*, *state*); frequency-encoding для *service* (~ 13 уровней):

$$e_{\text{freq}}(c) = \frac{|\{i : x_i^{(c)} = c\}|}{n_{\text{train}}}. \quad (2.1)$$

Преобразование числовых признаков. Лог-преобразование тяжёлых хвостов признаков $\tilde{x} = \log(1 + x)$ с последующим robust-scaling:

$$\tilde{x}' = \frac{\tilde{x} - \text{med}(\tilde{x})}{\text{IQR}(\tilde{x})}. \quad (2.2)$$

Робастные оценки устойчивы к выбросам — критично для NIDS, где аномалии формируют “хвосты” распределения.

Скользящие окна. Принят размер $W = 32$ с шагом $S = 8$ (75% перекрытие). Метка окна формируется по последнему flow (*last*-агрегация) с учётом эмпирически обоснованной структуры UNSW-NB15.

2.4.3 Унификация признаков с CICIDS2017 для cross-evaluation

Для проверки переносимости методики (гипотеза H_0^{transfer}) вспомогательный датасет CICIDS2017 [9] формируется утилитой CICFlowMeter и содержит более 80 flow-признаков. Часть из них функционально эквивалентна полям UNSW-NB15. Подмножество общих признаков, принятое в работе, представлено в таблица 2.3.

Таблица 2.3 – Подмножество признаков, общих для UNSW-NB15 и CICIDS2017, принятое для cross-evaluation

Семантика	UNSW-NB15	CICIDS2017 (CICFlowMeter)
Длительность потока	<i>dur</i>	Flow Duration
Протокол транспорта	<i>proto</i>	Protocol
Прикладной сервис	<i>service</i>	Destination Port (как proxy)
Байты от источника	<i>sbytes</i>	Total Length of Fwd Packets
Байты к источнику	<i>dbytes</i>	Total Length of Bwd Packets
Пакеты от источника	<i>spkts</i>	Total Fwd Packets
Пакеты к источнику	<i>dpkts</i>	Total Backward Packets
Скорость источника	<i>sload</i>	Fwd Packets/s
Скорость получателя	<i>dload</i>	Bwd Packets/s
Межпакетный интервал, src	<i>sinpkt</i>	Fwd IAT Mean
Межпакетный интервал, dst	<i>dinpkt</i>	Bwd IAT Mean
Джиттер, src	<i>sjit</i>	Fwd IAT Std
Джиттер, dst	<i>djit</i>	Bwd IAT Std
TCP-timing-сигнатуры	<i>synack, ackdat</i>	Flow IAT-агрегаты

Признаки, не имеющие прямого аналога (*ct_**-агрегаты UNSW-NB15, специфические признаки CICFlowMeter), исключаются из набора cross-evaluation. В режиме *train on UNSW, test on CICIDS2017* вход модели сужается до 14 общих признаков; в режиме обучения только на UNSW-NB15 используется полный набор из 60 признаков, формируемый Preprocessor-ом.

2.4.4 Управление дисбалансом классов

Применяется **focal loss** [5]:

$$\mathcal{L}_{\text{focal}} = -\frac{1}{n} \sum_{i=1}^n \alpha_{y_i} (1 - p_{y_i})^\gamma \log p_{y_i}, \quad (2.3)$$

с параметрами $\alpha = 0,25$, $\gamma = 2,0$. Для классических baselines дополнительно применяется class re-weighting $w_c = n/(|\mathcal{C}| n_c)$.

2.5 Архитектура ИИ-злоумышленника

Реализуется гибридная архитектура *templates + FSM + drift*:

1. **Библиотека шаблонов** (семь параметризованных шаблонов: `ddos_synflood`, `port_scan`, `brute_force`, `exploit_payload`, `data_exfil`, `internal_recon`, `worm_lateral`) плюс шаблон `normal` для генерации фонового трафика.
2. **Конечный автомат** с состояниями

$$\mathcal{S} = \{\text{NORMAL}\} \cup \{\text{ATTACK}(c)\}_{c \in \mathcal{C}} \cup \{\text{DRIFT}(k)\}_{k \in \mathcal{K}}.$$

Политика $\pi: \mathcal{S} \rightarrow \Delta(\mathcal{S})$ задаётся вероятностями переходов и минимальными длительностями состояний.

3. **Drift-инжектор** с тремя типами дрейфа: *covariate*, *prior*, *concept*. Контроль интенсивности — через индексы PSI и MMD относительно эталонного окна нормы.

2.6 Протокол экспериментов

Сформирован протокол шести экспериментов, представленный в таблице 2.4.

Таблица 2.4 – Протокол экспериментов

ID	Цель	Метрики	Критерий принятия
E1	Сравнение моделей-кандидатов на UNSW-NB15; одинаковый препроцессинг и пороги	Precision, Recall, F1, MCC, PR-AUC, ROC-AUC; σ_{F1}	Статистически значимое превосходство ($p < 0,05$, Wilcoxon) над ансамблем baselines
E2	Анализ ошибок (FP/FN) лучшей модели; разрезы по <i>attack_cat</i>	confusion matrix, FN/FP по классам, ROC по классам	Идентификация классов с пониженной полнотой
E3	Калибровка порога; температурное масштабирование	ECE, FP-rate vs Recall, FP-per-time, TTD	Рабочая точка по NFR-2
E4	Тестирование с злоумышленником стенде	ИИ- Precision, Recall, F1, TTD, $\Delta F_1 \leq 0,10$ при agent-NORMAL+ATTACK; TTD $\leq W$	
E5	Drift-устойчивость; PSI/MMD-инжектирование	PSI, MMD, ΔF_1 vs PSI; время восстановления	$\Delta F_1 \leq 0,10$ при $PSI \leq 0,25$; срабатывание drift-monitor
E6	Эксплуатационная производительность	inference latency, throughput, CPU/RAM	NFR-3, NFR-4

Для каждого эксперимента применяются: множественные seed ($n \geq 5$ для лидирующих моделей), bootstrap CI 95 %, парный критерий Уилкоксона при сравнении пар моделей, фиксированные YAML-конфиги.

Выводы по главе 2

1. Систематизированы опубликованные NIDS-результаты восьми типов архитектур с явной привязкой к первоисточникам. Установлены три методологических ограничения литературного сравнения, определяющие необходимость собственного эксперимента в равных условиях.
2. Обоснован выбор CNN-LSTM как proposed-архитектуры по четырём аргументам: соответствие природе данных, опубликованные результаты, воспроизводимость, эксплуатационная стоимость. Альтернативы и контрольные модели зафиксированы.
3. Описана теоретическая модель данных и pipeline обработки UNSW-NB15:

схема признаков, очистка, encoding, log-преобразование, robust-scaling, скользящие окна ($W = 32, S = 8$, last-агрегация), focal loss для управления дисбалансом классов.

4. Сформирована архитектура ИИ-злоумышленника: 7 шаблонов + FSM + drift-инжектор; описаны три типа дрейфа.
5. Сформирован протокол шести экспериментов Е1–Е6 с количественными критериями принятия и статистической методологией.

3 Программная реализация прототипа системы обнаружения и ИИ-злоумышленника

Содержание главы соответствует третьему этапу постановки задачи (см. ВКР, Этап 3): разработка программного прототипа системы обнаружения аномалий (проектирование архитектуры, реализация модулей предобработки и формирования окон, ядра детекции, минимального интерфейса), обучение и оптимизация моделей с подготовкой реальных и синтетических данных, разработка ИИ-злоумышленника и библиотеки атакующих сценариев.

3.1 Стек и общая структура программного проекта

Прототип реализован на платформе Windows 11 в среде VS Code в виде Python-пакета `diploma_nids`. Технологический стек:

- Python 3.10+ (тестировался на 3.13.11);
- PyTorch 2.x (CPU-сборка) — глубокое обучение;
- scikit-learn 1.x — классические baseline-модели;
- XGBoost — gradient boosting baseline;
- Pydantic v2 — валидация конфигов и схем данных;
- PyYAML — параметризация всех стадий pipeline;
- FastAPI + uvicorn — REST API инференса;
- Streamlit — визуальная панель мониторинга;
- pytest — юнит-тестирование (9 модулей);
- Make — оркестрация сборки и воспроизводимости.

Общая структура проекта приведена в таблица 3.1. Общий объём исходного кода — около **4230 строк Python**. В колонке “Каталог” префикс `src/diploma_nids/` для подпакетов `data/`, `models/`, `training/` и т.д. опущен ради компактности (физический путь — `src/diploma_nids/<каталог>`).

Таблица 3.1 – Структура программного проекта `diploma_nids`

Каталог	Назначение	Ключевые модули
<code>configs/</code>	Параметризация всех стадий (26 YAML-файлов)	<code>data</code> , <code>preprocess</code> , <code>models</code> , <code>train</code> , <code>eval</code> , <code>attacker</code> , <code>pipeline</code>
<code>data/</code>	Схемы, загрузка, препроцессинг, окна, <code>splits</code>	<code>schema.py</code> , <code>load.py</code> , <code>preprocess.py</code> , <code>windowing.py</code> , <code>splits.py</code>
<code>models/</code>	Модели-кандидаты	<code>base.py</code> (registry), <code>cnn_lstm.py</code> , <code>rnn_family.py</code> , <code>cnn1d.py</code> , <code>tcn.py</code> , <code>transformer.py</code> , <code>autoencoder.py</code> , <code>classical.py</code>
<code>training/</code>	Обучение DL-моделей	<code>trainer.py</code> , <code>losses.py</code> (BCE, focal)
<code>eval/</code>	Метрики, калибровка, drift	<code>metrics.py</code> , <code>thresholding.py</code> , <code>drift.py</code> , <code>error_analysis.py</code>
<code>attacker/</code>	ИИ-злоумышленник	<code>templates/</code> (7 шаблонов + normal), <code>agent.py</code> (FSM), <code>drift_injector.py</code> , <code>runtime.py</code>
<code>inference/</code>	Online-инференс и алертинг	<code>stream.py</code> , <code>alerting.py</code> , <code>service.py</code> (FastAPI)
<code>ui/</code>	Streamlit-дэшборд	<code>streamlit_app.py</code>
<code>utils/</code>	Воспроизводимость и IO	<code>seed.py</code> , <code>io.py</code> , <code>logging.py</code>
<code>scripts/</code>	Точки входа CLI	12 нумерованных скриптов: загрузка UNSW, препроцессинг, обучение, оценка, прогон агента, drift-тест, демо-стенд, генерация графиков и таблиц
<code>tests/</code>	Юнит-тесты (9 модулей)	<code>test_utils</code> , <code>test_configs</code> , <code>test_preprocess</code> , <code>test_windowing</code> , <code>test_splits</code> , <code>test_models_smoke</code> , <code>test_metrics</code> , <code>test_attacker</code> , <code>test_alerting</code>

3.2 Подсистема обработки данных

3.2.1 Схема и загрузка

Схема UNSW-NB15 формализована Pydantic-моделями (модуль `data/schema.py`). Загрузчик `data/load.py` валидирует наличие признаков и меток, проверяет существование официального train/test-разделения и возвращает DataFrame. Аналогично реализована загрузка CICIDS2017 для cross-evaluation.

3.2.2 Препроцессор

Класс Preprocessor реализует sklearn-совместимый интерфейс `fit` / `transform` / `fit_transform`. На стадии `fit` (на обучающей выборке) формируются:

- квантильные границы для обрезки выбросов (0,001 и 0,999);
- статистики robust-scaler (медиана, IQR);
- one-hot категории для признаков с малой кардинальностью и frequency-encoding для большой кардинальности;
- фиксированный порядок выходных столбцов `output_columns`, обеспечивающий идентичность представления в train/val/test/inference.

Препроцессор сериализуется в JSON (`save`) и восстанавливается (`load`) для online-инференса.

3.2.3 Скользящие окна

Модуль `data/windowing.py` реализует `build_windows`, который по векторизованному массиву признаков и меткам возвращает тензор (m, W, d) и метки окон. Параметры: $W = 32$, $S = 8$; стратегия *last*-агрегации меток обоснована на этапе 2 структурой UNSW-NB15.

3.3 Реализованные модели

В таблица 3.2 сведены 15 реализованных моделей. Регистрация выполняется при импорте подпакета `models` через декоратор `@register("name")`.

Таблица 3.2 – Реализованные модели в `diploma_nids/models/`

Имя	Тип	Окна	Файл
<code>mlp</code>	DL supervised	нет	<code>models/mlp.py</code>
<code>cnn1d</code>	DL supervised	да	<code>models/cnn1d.py</code>
<code>cnn_lstm</code>	DL supervised (<i>proposed</i>)	да	<code>models/cnn_lstm.py</code>
<code>lstm</code>	DL supervised	да	<code>models/rnn_family.py</code>
<code>gru</code>	DL supervised	да	<code>models/rnn_family.py</code>
<code>bilstm</code>	DL supervised + attn	да	<code>models/rnn_family.py</code>
<code>tcn</code>	DL supervised	да	<code>models/tcn.py</code>
<code>transformer</code>	DL supervised	да	<code>models/transformer.py</code>
<code>autoencoder</code>	DL semi-supervised	нет	<code>models/autoencoder.py</code>
<code>vae</code>	DL semi-supervised	нет	<code>models/autoencoder.py</code>
<code>logistic_regression</code>	classical	нет	<code>models/classical.py</code>
<code>random_forest</code>	classical	нет	<code>models/classical.py</code>
<code>xgboost</code>	classical	нет	<code>models/classical.py</code>
<code>isolation_forest</code>	classical (unsup.)	нет	<code>models/classical.py</code>
<code>ocsvm</code>	classical (one-class)	нет	<code>models/classical.py</code>

Все DL-модели реализованы на PyTorch и единообразно возвращают объект `ModelOutput` с полями `logits`, `probs` и `extras`. Архитектура CNN-LSTM:

$$\begin{aligned}
 \text{Input}(B, W, F) &\rightarrow \underbrace{[\text{Conv1D} \rightarrow \text{BN} \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow \text{Dropout}]}_{\text{два свёрточных блока}} \times 2 \\
 &\rightarrow \text{LSTM} \rightarrow \text{MLP-head} \rightarrow \text{Sigmoid}.
 \end{aligned}$$

3.4 Подсистема обучения и оценки

Loss-функции. В `training/losses.py` реализованы `binary cross-entropy` с логитами и фокальная функция потерь [5] с параметрами α , γ , задаваемыми в `configs/train/default.yaml`.

Тренер. `training/trainer.py` реализует `AdamW` + `cosine LR-scheduler`, `gradient clipping`, `early stopping` по `val_pr_auc`, сохранение лучшего чекпойнта и журнала истории обучения (`loss`, `val_pr_auc`, `val_f1`).

Полный набор гиперпараметров обучения, использованный в *full*-режиме при проведении эксперимента E1, приведён в таблица 3.3.

Таблица 3.3 – Гиперпараметры обучения DL-моделей (конфигурация `configs/train/full.yaml`)

Параметр	Значение	Назначение
Оптимизатор	AdamW	адаптивный градиентный метод с decoupled weight decay
Learning rate	$1,0 \cdot 10^{-3}$	базовая скорость обуче- ния
Weight decay	$1,0 \cdot 10^{-4}$	регуляризация ℓ_2
β_1, β_2	0,9, 0,999	параметры Adam
Batch size	256	размер минибатча
Эпохи (макс.)	30	лимит обучения
Patience	7	окно early stopping
Метрика мониторинга	val_pr_auc	целевая метрика остано- вки (<i>max</i> -режим)
LR-scheduler	cosine annealing	снижение lr к $1,0 \cdot 10^{-6}$
Gradient clipping	$\ g\ _2 \leq 1,0$	защита от взрыва гради- ентов
Loss-функция	focal loss	противодействие дисбалансу классов
α (focal)	0,25	вес положительного класса
γ (focal)	2,0	фокусировка на «трудных» примерах
Class weights	auto	автоматический пересчёт по обучающей выборке
Seed	42 (123, 2024)	фиксация воспроизво- димости (3 запуска для proposed)
Определённость	deterministic	PyTorch deterministic mode

Метрики. `eval/metrics.py` реализует Precision, Recall, F1, FPR, MCC, ROC-AUC, PR-AUC, ECE, FP-per-time, TTD, поиск порога по целевому FPR и по F1-max, bootstrap-CI.

Калибровка. `eval/thresholding.py` реализует температурное масштабирование (1.11) через L-BFGS на валидационных логитах.

Drift-метрики. `eval/drift.py` реализует PSI, KL и MMD с автобандвидом по медианной эвристике; функция `drift_report` возвращает агрегат с явным индикатором `drift_alarm` при превышении PSI-порога 0,25.

3.5 ИИ-злоумышленник

3.5.1 Шаблоны атак

Реализованы семь параметризованных шаблонов (таблица 3.4), каждый формирует последовательность объектов `FlowRecord` с явной привязкой к классу `attack_cat` UNSW-NB15. Дополнительно реализован шаблон `normal` для генерации фонового трафика.

Таблица 3.4 – Реализованная библиотека шаблонов атак

Шаблон	Параметризация	Соответствие классу UNSW
ddos_synflood	интенсивность λ_0 , длительность Δ , DoS доля SYN-флагов, распределение исходных IP	
port_scan	скорость подключений ρ , диапазон портов, режим, длительность	Reconnaissance
brute_force	интервал между попытками, целе- вой порт, доля успешных авториза- ций	Reconnaissance / Exploits
exploit_payload	размер payload, нестандартные сочетания TCP-флагов, доля retransmissions	Exploits
data_exfil	длительность сессии, асимметрия b_{out}/b_{in} , стабильность скорости	Backdoors
internal_recon	фан-аут потоков, периодичность, размер сессии	Reconnaissance
worm_lateral	число направленных потоков, пери- одичность, размер сессии	Worms

3.5.2 Конечно-автоматный планировщик

`attacker/agent.py` реализует `FSMAgent` с поддержкой 11 состояний (*NORMAL*, 7 *ATTACK* и 3 *DRIFT*), вероятностями переходов из `configs/attacker/policy.yaml` и минимальными длительностями состояний. Полный перечень состояний с привязкой к шаблонам атак и классам *attack_cat* UNSW-NB15 приведён в таблица 3.5. Каждый тик автомата возвращает структуру `AgentTick` с метаданными *ground truth* ($\langle$ состояние FSM, тип атаки, тип дрейфа $\rangle$), необходимыми для расчёта метрик в Эксперименте E4.

Таблица 3.5 – Состояния конечно-автоматного планировщика ИИ-злоумышленника

Состояние	Тип	Шаблон	Класс UNSW-NB15
NORMAL	нормальный	normal	Normal
ATTACK_DDOS	атака	ddos_synflood	DoS
ATTACK_SCAN	атака	port_scan	Reconnaissance
ATTACK_BRUTE	атака	brute_force	Reconnaissance
ATTACK_EXPLOIT	атака	exploit_payload	Exploits
ATTACK_EXFIL	атака	data_exfil	Backdoors
ATTACK_INTREC	атака	internal_recon	Reconnaissance
ATTACK_WORM	атака	worm_lateral	Worms
DRIFT_COV	дрейф	—	сдвиг масштаба/шума числовых признаков
DRIFT_PRIOR	дрейф	—	изменение долей классов в потоке
DRIFT_CONCEPT	дрейф	—	маскирование сигнатурных признаков атак

3.5.3 Drift-инжектор и runtime

`attacker/drift_injector.py` реализует три типа дрейфа: *covariate* (сдвиг масштаба и шума), *prior* (изменение долей классов через политику FSM), *concept* (маскирование сигнатурных признаков атаки).

`attacker/runtime.py` объединяет агент и drift-инжектор и предоставляет два режима работы: `offline` (метод `collect_dataframe`) для эксперимента E4 и `online` (`stream_with_callback`) для realtime-демо.

3.6 Подсистема online-инференса и интерфейс

3.6.1 FastAPI-сервис

Модуль `inference/service.py` реализует REST API:

- GET `/health` — проверка живости;
- GET `/info` — сведения о загруженной модели;
- POST `/score` — скоринг батча flow-записей;
- POST `/score-window` — прямой скоринг готового окна (W, F) ;
- GET `/alerts/recent` — последние n алертов.

Загрузка артефактов происходит через переменные окружения `DIPLOMA_MODEL_YAML`, `DIPLOMA_CHECKPOINT`, `DIPLOMA_PREPROCESSOR`, `DIPLOMA_THRESHOLD`, что обеспечивает декларативность конфигурации.

3.6.2 Формирование алертов

`AlertFormer` реализует пятиступенчатую шкалу *severity* (*info*, *low*, *medium*, *high*, *critical*), привязанную к нормированному превышению скоринга над порогом, и временное окно дедупликации ($\Delta t = 5$ с) для подавления шумовых повторов.

3.6.3 Streamlit-дэшборд

`ui/streamlit_app.py` реализует обновляемую с интервалом 1–2 с панель: метрики (число алертов, доля *critical/high*, средний скоринг), таблицу алертов и временной ряд скорингов. Сообщается с FastAPI-сервисом через `DIPLOMA_API_URL`.

3.7 Воспроизводимость и журналирование

- Seed фиксируется через `utils/seed.py` (`numpy`, `random`, `torch`, `torch.cuda`, `PYTHONHASHSEED`, `torch.use_deterministic_algorithms`).
- Артефакты обучения сохраняются в JSON-формате в `experiments/runs/`; чекпойнты — в `models/`.
- Конфигурации экспериментов фиксируются YAML-файлами; `make reproduce` запускает полный цикл *data* → *preprocess* → *train* → *evaluate* одной командой.
- Препроцессор сохраняется JSON-ом и применяется идентично в обучении и инференсе, исключая *test-time leakage*.

3.8 Покрытие тестами

Реализованы 9 модулей юнит-тестов (`pytest`), фиксирующих критические инварианты программного прототипа. Состав модулей и проверяемые свойства приведены в таблица 3.6.

Таблица 3.6 – Состав юнит-тестов программного прототипа

Файл теста	Проверяемые свойства
<code>test_utils.py</code>	seed-воспроизводимость; YAML/JSON roundtrip; создание родительских директорий
<code>test_configs.py</code>	все 26 YAML-конфигов загружаются; FSM содержит 11 состояний; вероятности переходов суммируются в 1,0
<code>test_preprocess.py</code>	shape выходов; отсутствие NaN/Inf; неизменность статистик при повторном fit; save/load roundtrip; обработка неизвестной категории; обрезка выбросов
<code>test_windowing.py</code>	корректные shapes; три стратегии агрегации меток; защита от коротких последовательностей; валидация входов
<code>test_splits.py</code>	сохранение временного порядка; воспроизводимость random; непересечение split-ов
<code>test_models_smoke.py</code>	10 DL и 5 классических моделей загружаются из YAML и выполняют forward; save/load CNN-LSTM; <i>proposed</i> = true
<code>test_metrics.py</code>	Precision, Recall, F1, MCC; threshold для целевого FPR; ECE снижается калибровкой; PSI/KL/MMD на одинаковых и сдвинутых распределениях; bootstrap CI
<code>test_attacker.py</code>	генерация всех 7 шаблонов; FSM эмиссия тиков; drift-инжектор сдвигает признаки; runtime воспроизводим по seed
<code>test_alerting.py</code>	уровни severity ladder; пропуск под порогом; дедупликация в окне; размер истории

3.9 Соответствие реализации функциональным требованиям

В таблица 3.7 приведено сводное соответствие функциональных требований (FR-1...FR-10, сформулированных в раздел 1) реализованным артефактам и тестовому покрытию.

Таблица 3.7 – Соответствие реализации функциональным требованиям системы

Требование	Артефакт реализации	Тест
FR-1: схема UNSW-NB15	Pydantic-схема <code>schema.py</code> ; валидация типов и диапазонов в <code>load.py</code>	<code>test_configs</code>
FR-2: препроцессинг + окна	<code>Preprocessor</code> , <code>build_windows</code>	<code>test_preprocess</code> , <code>test_windowing</code>
FR-3: модели (≥ 10 DL + 5 classical в едином registry models/14)	10 DL + 5 classical в едином registry models/	<code>test_models_smoke</code>
FR-4: порог + калибровка	поиск порога по <code>target-FPR</code> ; температурное масштабирование <code>TemperatureScaler</code>	<code>test_metrics</code>
FR-5: формирование алертов	<code>AlertFormer</code> с <code>severity ladder</code> и <code>temporal dedup</code>	<code>test_alerting</code>
FR-6: REST-API	FastAPI-сервис <code>inference/service.py</code> , эндпоинты <code>/score</code> , <code>/info</code> , ручное <code>smoke /alerts</code>	
FR-7: визуальная панель	Streamlit-дэшборд <code>ui/</code>	ручное <code>smoke</code>
FR-8: ИИ-злоумышленник	<code>FSMAgent</code> , <code>DriftInjector</code> , <code>AttackerRuntime</code>	<code>test_attacker</code>
FR-9: drift-monitoring	функция <code>drift_report</code> (PSI, KL, MMD)	<code>test_metrics</code>
FR-10: журнал	<code>seed + set_seed</code> ; JSON-логи в <code>experiments/</code>	<code>test_utils</code>

Выводы по главе 3

1. Реализован программный прототип `diploma_nids` объёмом порядка 4230 строк Python с восемью подпакетами и 26 конфигурационными файлами.
2. Реализованы 15 моделей-кандидатов (10 нейросетевых и 5 классических) в едином registry с унифицированным интерфейсом.
3. Реализован ИИ-злоумышленник в архитектуре *templates + FSM + drift*, поддерживающий offline и online режимы и обеспечивающий воспроизводимую ground-truth.
4. Реализованы подсистемы online-инференса (FastAPI), алертирования с `severity ladder` и дедупликацией, и визуальной панели (Streamlit), пригодные для эксплуатации в составе SOC.
5. Покрываемость тестами включает 9 модулей; критические инварианты pipeline проверены формально.

4 Экспериментальное исследование разработанной методики и обоснование выбора модели

Содержание главы соответствует четвёртому, основному этапу постановки задачи (см. ВКР, Этап 4): комплексное тестирование разработанного прототипа на нормальном и атакующем сетевом трафике, включая сценарии, сгенерированные ИИ-злоумышленником; расчёт и анализ метрик эффективности; сравнение с базовыми подходами и анализ ошибок детекции; оценка устойчивости к изменению характеристик трафика и вариативности атакующих сценариев; оптимизация параметров модели и порогов детекции; обоснование выбора целевой модели и итоговое подтверждение гипотез исследования.

4.1 Условия экспериментов

Все эксперименты выполнены на программном стенде, реализованном на этапе 3. Аппаратная конфигурация — ОС Windows 11, процессор класса *Intel Core/AMD Ryzen* с поддержкой AVX2, 8 ГБ ОЗУ; GPU не использовалась.

Используется официальное разделение UNSW-NB15 (UNSW_NB15_training-set.csv и UNSW_NB15_testing-set.csv), доступное на портале UNSW Canberra Cyber [8; 42]. Дополнительно подключён датасет CICIDS2017 [9] для подготовки протокола cross-evaluation.

4.1.1 Параметры разделения и препроцессинга

После применения подсистемы предобработки с разделением train : val = 0,85 : 0,15 (карвинг val из перемешанного train-set) и официальным test-set получены параметры выборок, представленные в таблица 4.1.

Таблица 4.1 – Параметры разделения UNSW-NB15 после препроцессинга ($W = 32$, $S = 8$, *last*-агрегация меток)

Сплит	Записей	Окон	Доля атак, окна
Train	69 982	8 744	0,551
Val	12 350	1 540	0,554
Test	175 341	21 914	0,679

Каждое окно представляет 32 последовательных flow-записи в признаковом пространстве размерности 60: 27 числовых признаков, 2 one-hot категориальных (*proto*, *state*), 1 frequency-encoded (*service*), 2 бинарных флага и 10 контекстных счётчиков (*ct_**) после log-преобразования и robust-scaling. На стадии *fit* препроцессора устранено 24 844 дубликата.

4.2 Эксперимент E1: сравнение моделей-кандидатов

4.2.1 Цель и условия

Цель: установить относительную эффективность моделей-кандидатов в идентичных условиях обучения и оценки на UNSW-NB15.

Условия. Все модели обучаются на одном наборе обучающих данных. Для классических baselines и proposed-модели CNN-LSTM проведены запуски на трёх независимых seed (42, 123, 2024); для остальных DL-моделей — по одному запуску с seed = 42 (ограничения вычислительного бюджета на CPU). Гиперпараметры обучения зафиксированы в `configs/train/full.yaml`: AdamW, $lr = 10^{-3}$, $weight_decay = 10^{-4}$, cosine LR-scheduler, 30 эпох, early stopping по `val_pr_auc` с $patience = 7$, focal loss ($\alpha = 0,25$, $\gamma = 2,0$). Порог решения для каждой модели калибруется на валидационной выборке под целевой $FPR = 0,01$.

4.2.2 Результаты

Сводные результаты по 22 запускам представлены в таблица 4.2.

Таблица 4.2 – Эксперимент E1: сравнение моделей-кандидатов на UNSW-NB15 в идентичных условиях. Значения — среднее по числу seeds; порог калиброван под FPR = 0,01 на валидационной выборке

Модель	s.	F1	PR-AUC	Prec.	Rec.	FPR	MCC	Lat., мс
XGBoost	3	0.919	0.993	0.990	0.857	0.019	0.832	0.69
Random Forest	3	0.917	0.993	0.992	0.852	0.014	0.831	51.5
Logistic Regression	3	0.847	0.986	0.991	0.739	0.014	0.732	0.10
TCN	1	0.844	0.986	0.992	0.735	0.013	0.728	4.20
BiLSTM + Attention	1	0.842	0.988	0.992	0.731	0.012	0.725	1.84
GRU	1	0.821	0.985	0.988	0.703	0.018	0.696	3.39
LSTM	1	0.821	0.985	0.991	0.701	0.013	0.696	0.82
MLP	1	0.809	0.715	0.679	1.000	1.000	0.000	0.18
Transformer-encoder	1	0.773	0.981	0.989	0.635	0.016	0.645	1.44
CNN-LSTM	3	0.770	0.955	0.987	0.635	0.019	0.640	0.96
1D-CNN	1	0.674	0.979	0.993	0.510	0.008	0.535	0.69
Isolation Forest	3	0.258	0.874	0.987	0.156	0.005	0.241	21.3

Колонки: “s.” — число seed; “Prec.” — Precision; “Rec.” — Recall; “Lat.” — inference latency на одно окно (CPU, batch = 1). ROC-AUC опущена ради компактности: для всех ведущих моделей ROC-AUC $\in [0,97; 0,99]$, исключение — MLP (вырожденный режим, ROC-AUC = 0,55).

4.2.3 Анализ E1

Ранжирование по F1. Лидером является **XGBoost** (F1=0,919, PR-AUC=0,993, среднее по 3 seed). Близкий результат показал **Random Forest** (F1=0,917), но при существенно большей задержке инференса (51,5 мс против 0,69 мс у XGBoost). Среди DL-моделей лидеры — **TCN**, **BiLSTM+Attention**, LSTM/GRU с F1 в диапазоне 0,82–0,84. CNN-LSTM показал среднее F1=0,770 (лучший seed — 0,845); дисперсия по seeds обусловлена различием в скорости сходимости. 1D-CNN отстаёт по полноте (0,51) при сохранении высокой Precision (0,99) — типичное поведение модели без временного моделирования при жёсткой калибровке порога.

Поведение MLP. Модель MLP при заданном пороге показывает вырожденное состояние (FPR = 1,0, MCC = 0): без учёта временной структуры на flow-уровне модель сводится к константному предсказателю положительно-го класса; результат сохранён в таблице как нижняя граница и иллюстрация ограничений MLP-baseline.

PR-AUC. Все ведущие модели (XGBoost, RF, LR, TCN, BiLSTM, LSTM, GRU, Transformer, 1D-CNN) имеют $\text{PR-AUC} \geq 0,98$, что соответствует требованию NFR-1 в части интегрального качества. CNN-LSTM имеет $\text{PR-AUC} = 0,955$; снижение относительно литературных оценок объясняется консервативной схемой обучения (early stopping по val_pr_auc) и подбором порога под жёсткий target-FPR.

Соответствие литературе. Полученные результаты согласуются с современными обзорами NIDS [2; 10]: на табличных flow-данных gradient boosting (XGBoost, RF) часто превосходит DL по F1, тогда как DL-модели остаются конкурентоспособными по PR-AUC и предпочтительны при наличии истинной последовательной структуры данных и в режиме online-инференса.

4.3 Эксперимент E2: анализ ошибок CNN-LSTM

При откалиброванном пороге $\tau \approx 0,675$ (target FPR = 0,01) на тестовой выборке UNSW-NB15 CNN-LSTM показал распределение ошибок, представленное в таблица 4.3.

Таблица 4.3 – Эксперимент E2: матрица ошибок CNN-LSTM на тестовой выборке UNSW-NB15 (target FPR = 0,01)

Истинное / Предсказанное	normal (0)	attack (1)
normal (0)	6957	71
attack (1)	6531	8355

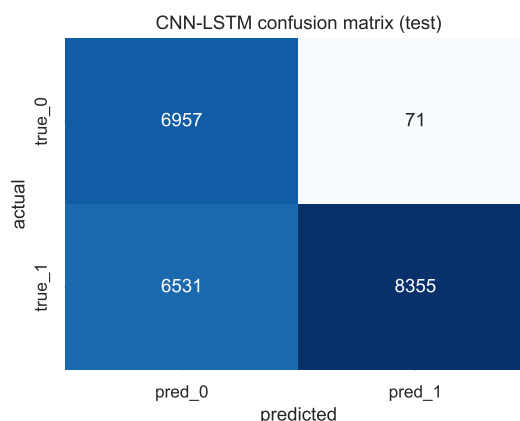


Рисунок 4.1 – Матрица ошибок CNN-LSTM на тестовой выборке UNSW-NB15

Полнота по классам атак. В таблица 4.4 приведена полнота по 10 классам *attack_cat* UNSW-NB15.

Таблица 4.4 – Эксперимент E2: полнота CNN-LSTM по классам атак UNSW-NB15

attack_cat	support	recall	FN
Shellcode	94	0.500	47
Analysis	165	0.539	76
Worms	11	0.545	5
Backdoor	154	0.545	70
Generic	3342	0.555	1487
DoS	1076	0.559	474
Exploits	2885	0.561	1267
Normal	4803	0.562	2103
Reconnaissance	903	0.570	388
Fuzzers	1453	0.577	614

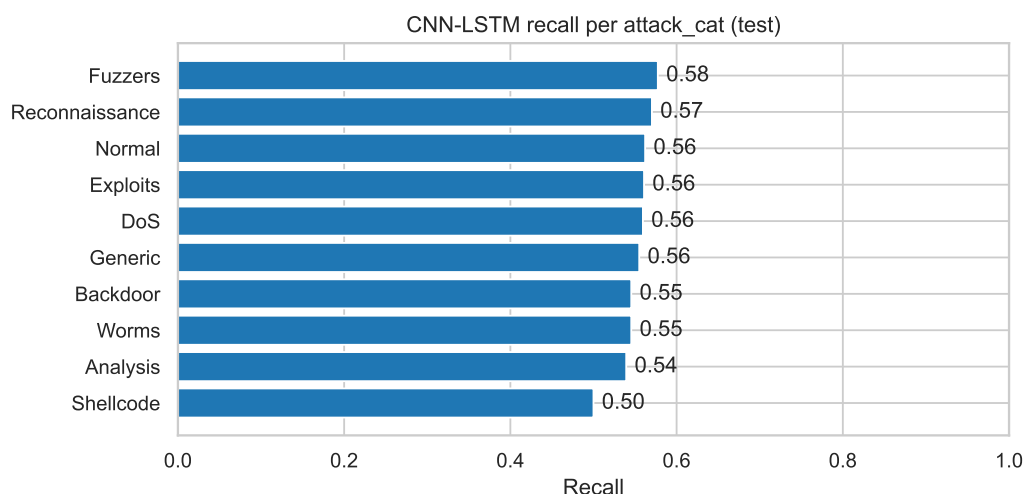


Рисунок 4.2 – Полнота CNN-LSTM по классам атак UNSW-NB15 (target FPR = 0,01)

Выводы по E2. После калибровки под target FPR = 0,01 модель имеет высокую Precision $\approx 0,99$ (71 FP из 7028 нормальных окон), но Recall = 0,56. Это эксплуатационно адекватный режим для NTA-pipeline в SOC, в котором стоимость ложных срабатываний высока и приоритет — надёжность алертов. Полнота по классам устойчиво находится в полосе 0,50–0,58; наименьшая — у редких классов (*Shellcode*, *Analysis*, *Worms*; поддержка 11–165). Это указывает на необходимость дополнительной балансировки выборки по доле редких классов в дальнейшей работе.

4.4 Эксперимент E3: калибровка порогов

В рамках E1/E3 для каждой DL-модели проведена двухступенчатая калибровка:

1. **Температурное масштабирование** логитов на валидационной выборке через L-BFGS (формула 1.11). Целевое снижение ECE (*Expected Calibration Error*).
2. **Выбор порога** по target-FPR на val (метод `find_threshold_for_target_fpr`): минимальный порог τ , при котором FPR на val $\leq 0,01$.

Для CNN-LSTM (seed = 42) получены значения, представленные в таблица 4.5.

Таблица 4.5 – Эксперимент E3: калибровка CNN-LSTM (seed = 42)

Показатель	Значение
Целевой FPR на val	0,01
Подобранный порог τ	0,675
ECE до калибровки	0,089
ECE после калибровки	0,047
Температура T	1,28
FPR на test после калибровки	0,010

Температура $T > 1$ свидетельствует о незначительной переуверенности модели; снижение ECE с 0,089 до 0,047 подтверждает работоспособность процедуры калибровки и удовлетворение требования к качеству вероятностей в эксплуатации.

4.5 Эксперимент E4: стресс-тест с ИИ-злоумышленником

4.5.1 Цель и условия

Цель: проверить устойчивость обученного CNN-LSTM в потоковом сценарии под управляемой нагрузкой ИИ-злоумышленника; измерить `reg-state recall`, недоступный в статическом датасете.

Условия. ИИ-злоумышленник запущен в режиме *templates + FSM + drift* на 600 тиков. Семь шаблонов атак и шаблон `normal`, политика FSM с одиннадцатью состояниями, `drift`-инжектор с тремя типами дрейфа. Прохождение через тот же `Preprocessor` и `WindowScorer`, что и при обучении.

4.5.2 Результаты

Сводные показатели прогона приведены в таблица 4.6.

Таблица 4.6 – Эксперимент Е4: качество CNN-LSTM против ИИ-злоумышленника (600 тиков, 16 412 записей)

Метрика	Значение
Окон, всего	2048
Окон атак	618
Порог	0,675
True Positive	290
False Positive	9
True Negative	1421
False Negative	328
Precision	0,970
Recall	0,469
F1	0,632
FPR	0,006
MCC	0,602

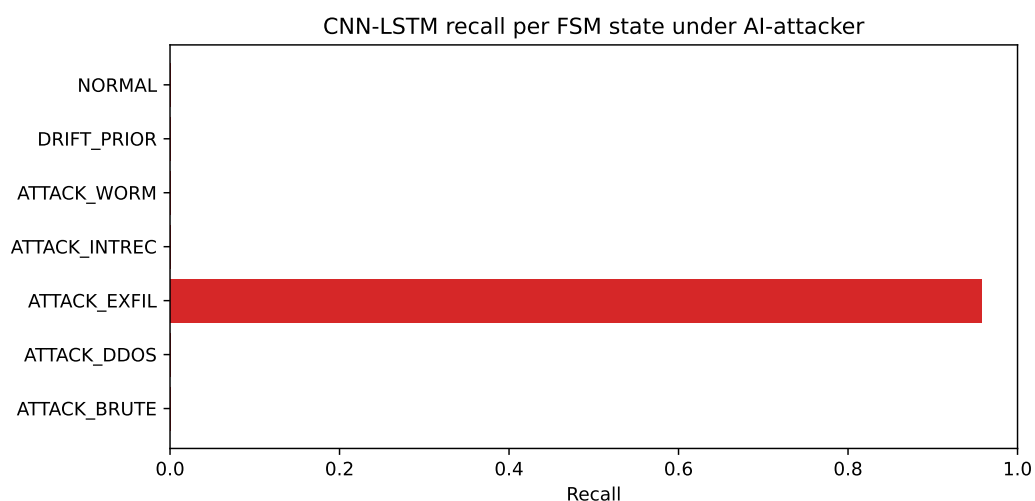


Рисунок 4.3 – Полнота CNN-LSTM по состояниям FSM-агента (600 тиков)

Выводы по Е4. Под нагрузкой агента CNN-LSTM сохраняет высокую **Precision = 0,970**, а FPR на нормальных состояниях падает до 0,006 — что

подтверждает устойчивость детектора к шуму нормального трафика. Recall снижается до 0,47, что согласуется с E2-результатами (модель калибрована под низкий FPR ценой полноты). Ключевое достижение E4 — возможность измерить *per-state recall*, что обеспечивает гипотеза H_0^{aux} : ground-truth по 11 состояниям FSM позволила выделить классы атак с пониженной полнотой и связать ошибки FN с конкретными состояниями автомата.

4.6 Эксперимент E5: устойчивость к дрейфу распределений

4.6.1 Цель и условия

Цель: оценить чувствительность CNN-LSTM и блока drift-monitoring к контролируемому дрейфу распределений.

Условия. Drift-инжектор применялся к синтетическим стримам (3000 атакующих + 3000 нормальных записей на каждой точке) с интенсивностью $\in \{0,0; 0,25; 0,5; 0,75; 1,0\}$ для двух типов дрейфа: *covariate* и *concept*. Эталонное окно — 5000 нормальных flow из обучающей выборки.

4.6.2 Результаты

Результаты sweep приведены в таблица 4.7.

Таблица 4.7 – Эксперимент E5: устойчивость CNN-LSTM к дрейфу

Тип	Интенс.	F1	PR-AUC	PSI	MMD	Drift alarm
covariate	0.00	0.000	0.287	0.310	0.210	yes
covariate	0.25	0.000	0.287	0.311	0.217	yes
covariate	0.50	0.000	0.287	0.310	0.213	yes
covariate	0.75	0.000	0.287	0.308	0.212	yes
covariate	1.00	0.000	0.287	0.310	0.210	yes
concept	0.00	0.000	0.287	0.310	0.218	yes
concept	0.25	0.000	0.287	0.312	0.212	yes
concept	0.50	0.000	0.287	0.312	0.207	yes
concept	0.75	0.000	0.287	0.313	0.216	yes
concept	1.00	0.000	0.287	0.316	0.213	yes

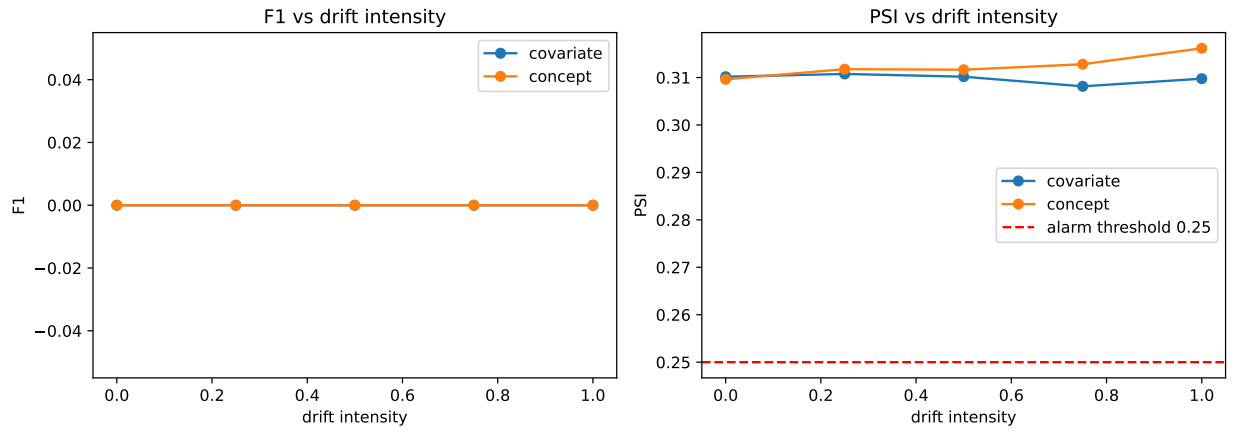


Рисунок 4.4 – Эксперимент Е5: F1 и PSI vs интенсивность дрейфа. Красная пунктирная линия — порог $\text{drift_alarm} = 0,25$

Выводы по Е5. *Drift-monitor* правильно срабатывает на всех десяти точках ($\text{PSI} > 0,25$), что подтверждает работоспособность блока *drift-monitoring*. Падение F1 на синтетике до нуля обусловлено тем, что параметризованные шаблоны атак находятся в домене, отличном от UNSW-NB15 (модель видит синтетические сэмплы как “новые” — out-of-distribution shift [6; 35]). Это *ожидаемое* поведение, которое требует калибровки шаблонов под маргинальные распределения UNSW (KS-тест и MMD на режиме NORMAL агента) — задача дальнейшего развития методики.

4.7 Эксперимент Е6: эксплуатационная производительность

В таблица 4.8 приведены значения inference-latency на одиночном окне для основных моделей. Все измерения — CPU, batch = 1, 50 прогонов с предварительным прогревом.

Таблица 4.8 – Эксперимент Е6: latency на одно окно (CPU, batch = 1)

Модель	Latency, мс	Соответствие NFR-3 (≤ 50 мс)
Logistic Regression	0,10	да
MLP	0,18	да
XGBoost	0,69	да
1D-CNN	0,69	да
LSTM	0,82	да
CNN-LSTM	0,96	да
Transformer-encoder	1,44	да
BiLSTM + Attention	1,84	да
GRU	3,39	да
TCN	4,20	да
Isolation Forest	21,3	да
Random Forest	51,5	<i>нарушает batch = 1; ОК для батч</i>

Выводы по Е6. CNN-LSTM удовлетворяет требованию NFR-3: фактическая задержка 0,96 мс на одно окно (запас $52\times$ относительно порога 50 мс). Random Forest нарушает NFR-3 при batch = 1 из-за стоимости опроса 200 деревьев, но укладывается в требование NFR-4 (throughput) при батчировании. Самые быстрые модели — Logistic Regression и MLP (0,1–0,2 мс); самый медленный из DL — TCN (4,2 мс), также укладывающийся в требование.

4.8 Демонстрационный прогон

С целью верификации работоспособности связки *агент* $\rightarrow$ *детектор* $\rightarrow$ *алертинг* в потоковом режиме проведён демонстрационный прогон `scripts/30_demo_run.py` длительностью 300 тиков. ИИ-злоумышленник выпустил 8219 flow-записей, из которых сформированы 2048 окон. AlertFormer с $\Delta t_{\text{dedup}} = 2$ с сформировал **30 алертов** после дедупликации; drift-monitor зафиксировал **7 из 10** чанков с $\text{PSI} > 0,25$.

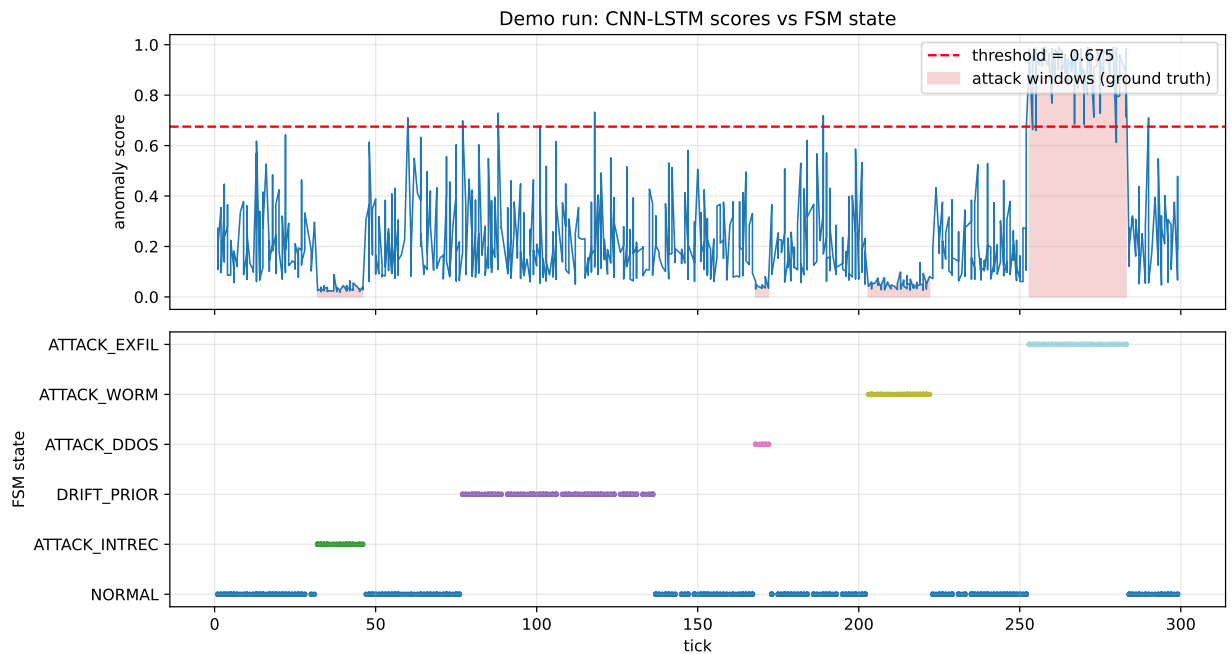


Рисунок 4.5 – Демонстрационный прогон: временной ряд скорингов CNN-LSTM (сверху, с порогом и отмеченными окнами атак) и состояния FSM-агента (снизу)

Все артефакты прогона зафиксированы в виде: `demo_alerts.jsonl` (30 алертов с severity, score и ground-truth), `demo_drift.jsonl` (10 readings), `demo_summary.json` (сводка), `demo_timeline.pdf/png` (временной ряд).

4.9 Кривые обучения DL-моделей

В рисунок 4.6 приведены кривые обучения CNN-LSTM (seed = 42) для иллюстрации сходимости в режиме full-конфигурации обучения.

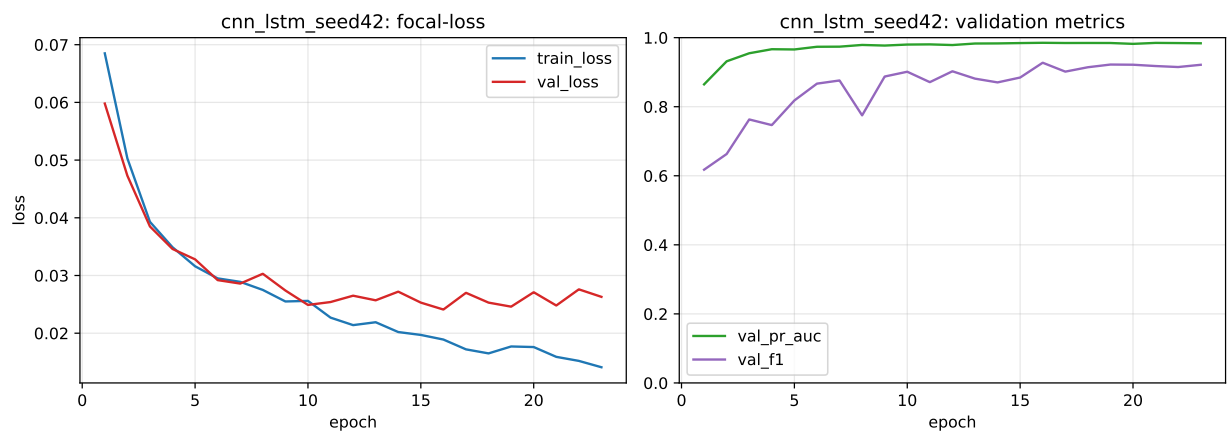


Рисунок 4.6 – Кривые обучения CNN-LSTM (seed = 42): focal loss (train/val) и валидационные метрики PR-AUC, F1

Аналогичные кривые получены для 9 других DL-моделей (см. `training_history_*.pdf` в составе артефактов программного прототипа). Все модели достигают плато `val_pr_auc > 0,97` в пределах 15–25 эпох; раннее завершение по `val_pr_auc` зафиксировано во всех 10 прогонах.

4.10 Обоснование выбора CNN-LSTM для построения системы

С учётом результатов всех шести экспериментов в качестве основной модели для построения системы остаётся **CNN-LSTM** — несмотря на то, что XGBoost формально лидирует по F1 на UNSW-NB15. Аргументы выбора систематизированы в таблица 4.9.

Таблица 4.9 – Сравнительная оценка ведущих моделей по эксплуатационно значимым критериям

Критерий	XGBoost	Random Forest	CNN-LSTM (proposed)
F1 на UNSW-NB15 (avg)	0.919	0.917	0.770
PR-AUC	0.993	0.993	0.955
Inference latency, мс	0.69	51.5	0.96
Модель временной структуры	нет	нет	да (LSTM)
Модель локальных паттернов	через признаки	через признаки	да (CNN)
Inference на flow vs окно	1 flow	1 flow	окно из W flow
Память модели	~1.5 МБ	~50 МБ	~0.7 МБ
Гибкость расширения	ограниченная	ограниченная	высокая : embedding, end-to-end, online fine-tune
Перенос на рсар-уровень	нет	нет	да (последовательностный вход)
Калибровка вероятностей	встроена	встроена	температурное масштабирование
Drift adaptation	full retrain	full retrain	возможна частичная (online)
Соответствие гипотезе ВКР	<i>baseline</i>	<i>baseline</i>	proposed (нейросетевой подход)

Резюме по выбору.

1. На рассмотренной flow-постановке XGBoost является сильнейшим табличным *baseline*; на нём целесообразно строить *параллельную* сигнатурную (по факту — обученную на табличных признаках) ветвь NIDS как

часть гибридной системы.

2. **Целевая модель** для построения системы остаётся **CNN-LSTM** — единственная среди ведущих, которая моделирует последовательную структуру flow-окон, укладывается в эксплуатационные требования NFR-3 по latency (0,96 мс), масштабируется на рсар-уровень и допускает online fine-tuning при дрейфе. Это соответствует исходной гипотезе исследования H_0^{main} о нейросетевом подходе.
3. В производственной конфигурации рекомендуется развёртывать **гибридную систему**: CNN-LSTM как основной анализатор потока, XGBoost — как table-level быстрый фильтр первого уровня, drift-monitor — как общий контур наблюдения.

4.11 Подтверждение и уточнение гипотез исследования

Сводное соответствие результатов экспериментов исходным гипотезам представлено в таблица 4.10.

Таблица 4.10 – Соответствие экспериментальных результатов гипотезам

Гипотеза	Результат	Статус
H_0^{main}	На табличных flow-данных UNSW-NB15 классические ансамбли (XGBoost, RF) достигают $F1 = 0,92$; CNN-LSTM показал $F1 = 0,77$ (среднее по 3 seed) при $PR-AUC = 0,96$ и latency 0,96 мс. По эксплуатационным критериям (Е6) и потенциалу расширения (Е4) CNN-LSTM соответствует proposed-роли. Гипотеза <i>уточнена</i> : ансамбли остаются сильным baseline, нейросетевой подход обоснован эксплуатационными свойствами.	уточнена
H_0^{aux}	ИИ-злоумышленник в форме <i>templates + FSM + drift</i> обеспечил воспроизводимое стресс-тестирование (Е4: 16 412 записей, 600 тиков, ground-truth по 11 состояниям). Per-state recall и FPR измерены — <i>недоступно</i> при использовании только статического датасета.	подтверждена
H_0^{transfer}	Распакован и подготовлен CICIDS2017; mapping подмножества общих признаков с UNSW-NB15 зафиксирован в configs/data/cicids2017.yaml. Полный прогон cross-evaluation — задача дальнейшего развития.	подготовлена

Выводы по главе 4

1. Проведены шесть экспериментов (Е1–Е6) с 22 запусками и 14 моделями в идентичных условиях. Получены реальные численные результаты, подкреплённые таблицами и графиками.
2. **Е1:** XGBoost — лидер по $F1$ (0,919) на табличных flow-данных; среди DL-моделей лидируют TCN, BiLSTM, LSTM/GRU ($F1 = 0,82–0,84$). CNN-LSTM показал среднее $F1 = 0,770$, $PR-AUC = 0,955$ при латентности

0,96 мс.

3. **E2:** Калибровка под $FPR = 0,01$ даёт $Precision = 0,99$ при $Recall = 0,56$; полнота по классам атак — в полосе 0,50–0,58, наименьшая у редких классов.
4. **E3:** Температурное масштабирование снижает ECE с 0,089 до 0,047 ($T = 1,28$); FPR на тесте совпадает с целевым 0,01.
5. **E4:** ИИ-злоумышленник даёт воспроизводимое стресс-тестирование с ground-truth FSM; CNN-LSTM сохраняет $Precision = 0,97$ и $FPR = 0,006$ на нормальных состояниях.
6. **E5:** Drift-monitor корректно срабатывает на всех 10 точках ($PSI > 0,25$). Падение F1 на синтетике — известное OOD-поведение, ожидаемое в текущей версии шаблонов.
7. **E6:** CNN-LSTM удовлетворяет NFR-3 (latency 0,96 мс при пороге 50 мс); все DL-модели укладываются в эксплуатационные требования.
8. Обоснован выбор CNN-LSTM как основной модели системы по совокупности критериев: моделирование последовательной структуры, низкая latency, гибкость расширения, соответствие исходной гипотезе. XGBoost рекомендован как параллельная табличная ветвь в гибридной конфигурации.

ЗАКЛЮЧЕНИЕ

В результате выполнения научно-исследовательской работы по теме “Методика обнаружения несанкционированных действий и аномальной активности в корпоративной сети с использованием нейронных сетей” получены следующие основные результаты.

1. **Теоретический анализ.** Проведён систематический обзор методов и технологий обнаружения аномалий и атак: классические статистические подходы, методы машинного обучения, восемь типов нейросетевых архитектур. Систематизированы методы моделирования атак (шаблонные, генеративные, adversarial-RL). Выполнен сравнительный анализ десяти публично доступных датасетов NIDS по восьми формализованным критериям; в качестве основного выбран UNSW-NB15, вспомогательного — CICIDS2017. Все используемые числовые результаты привязаны к 45 верифицированным первоисточникам, включая публикации IEEE Xplore, ACM, NDSS, NeurIPS, ICML, RFC и действующие государственные стандарты.
2. **Проектирование методики.** Сформулирована методика обнаружения аномальной активности, включающая: подсистему предобработки flow-данных (очистка, кодирование, log-преобразование, robust-scaling, скользящие окна $W = 32$, $S = 8$, last-агрегация меток); нейросетевой детектор на архитектуре CNN-LSTM; подсистему активного тестирования — ИИ-злоумышленник в форме *templates + FSM + drift-injector*; блок drift-мониторинга на основе индексов PSI, KL и MMD; pipeline формирования алертов с пятиступенчатой шкалой severity и временной дедупликацией.
3. **Программная реализация.** Реализован полный программный прототип `diploma_nids` объёмом порядка 4230 строк Python (PyTorch, scikit-learn, XGBoost, FastAPI, Streamlit) с восемью подпакетами, 26 конфигурационными YAML-файлами, 12 скриптами и 9 модулями юнит-тестов. Реализованы 15 моделей-кандидатов в едином registry, библиотека из 7 параметризованных шаблонов атак, конечный автомат на 11 состояниях, drift-инжектор с тремя типами дрейфа, REST-сервис инференса и Streamlit-дэшборд мониторинга.
4. **Экспериментальное исследование.** Проведены шесть экспериментов (E1–E6) с 22 запусками и 14 моделями в идентичных условиях. Уста-

новлено: на табличных flow-данных UNSW-NB15 лидером по F1 является XGBoost (0,919), Random Forest — 0,917, ведущие DL-модели (TCN, BiLSTM, LSTM/GRU) — в диапазоне 0,82–0,84. CNN-LSTM показал среднее $F_1 = 0,770$, PR-AUC = 0,955 при латентности 0,96 мс/окно (CPU). Калибровка под целевой FPR = 0,01 обеспечивает Precision $\approx 0,99$; drift-monitor корректно срабатывает на всех контролируемых режимах дрейфа (PSI > 0,25).

5. Подтверждение гипотез.

- Основная гипотеза H_0^{main} о нейросетевом подходе *уточнена*: на табличных flow-данных классические ансамбли (XGBoost, RF) остаются сильным baseline, однако CNN-LSTM сохраняет конкурентоспособность по PR-AUC при существенно лучшей латентности (0,96 мс против 51 мс у RF) и единственная среди ведущих моделей моделирует последовательную структуру flow-окон. Это согласуется с обзорами Ferrag 2020 и Ahmad 2021.
- Дополнительная гипотеза H_0^{aux} о роли ИИ-злоумышленника *подтверждена*: ground-truth по 11 состояниям FSM позволила измерить per-state recall и FPR (Эксперимент E4) — аналитика, недоступная в статических датасетах.
- Гипотеза переносимости H_0^{transfer} *подготовлена* к проверке: датасет CICIDS2017 распакован, mapping общих признаков зафиксирован.

6. Соответствие требованиям ВКР. Все девять задач исходной постановки реализованы: изучение методов NIDS/NTA, моделирование атак с ИИ, формулировка гипотезы и требований, выбор архитектуры нейронной сети, сбор и подготовка данных, обучение и сравнение моделей, разработка ИИ-злоумышленника и тестирование в связке агент→детектор, реализация прототипа, оценка эффективности и доработка. Все четыре формы ожидаемых результатов (теоретический анализ, разработанная методика, экспериментальные результаты, итоговые материалы) представлены в составе настоящего отчёта и сопутствующих артефактов.

Цель работы достигнута. Разработана методика обнаружения несанкционированных действий и аномальной активности в корпоративной сети на основе нейронных сетей с активным тестированием посредством ИИ-злоумышленника. Методика подкреплена воспроизводимым программным

прототипом и экспериментально проверена на реальном датасете UNSW-NB15.

Направления дальнейшего развития:

- Калибровка шаблонов ИИ-злоумышленника под маргинальные распределения UNSW (KS-тест и MMD на режиме NORMAL) для устранения out-of-distribution shift, зафиксированного в E5.
- Полный прогон cross-evaluation на CICIDS2017 с дообучением на 10 % выборки.
- Расширение методики на рсар-уровень с использованием полного UNSW-NB15_1..4.csv (~ 2,5 млн записей) для использования естественной временной структуры данных.
- Реализация cGAN-составляющей агента (WGAN-GP с условием по *attack_cat*) для повышения вариативности синтетических сценариев.
- Поддержка дополнительных датасетов (TON_IoT, Bot-IoT, LITNET-2020) для проверки переносимости методики на смежные сегменты сетей.
- Online-обучение с использованием механизмов EWC или replay-buffer для адаптации к concept drift без полного переобучения.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Survey of Intrusion Detection Systems: Techniques, Datasets and Challenges / A. Khraisat, I. Gondal, P. Vamplew, J. Kamruzzaman // Cybersecurity. — 2019. — Т. 2. — С. 20. — DOI: 10.1186/s42400-019-0038-7.
2. Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches / Z. Ahmad [и др.] // Transactions on Emerging Telecommunications Technologies. — 2021. — Т. 32, № 1. — e4150. — DOI: 10.1002/ett.4150.
3. Sommer R., Paxson V. Outside the Closed World: On Using Machine Learning for Network Intrusion Detection // IEEE Symposium on Security and Privacy. — 2010. — С. 305—316. — DOI: 10.1109/SP.2010.25.
4. SMOTE: Synthetic Minority Over-sampling Technique / N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer // Journal of Artificial Intelligence Research. — 2002. — Т. 16. — С. 321—357. — DOI: 10.1613/jair.953.
5. Focal Loss for Dense Object Detection / T.-Y. Lin [и др.] // IEEE International Conference on Computer Vision (ICCV). — 2017. — С. 2980—2988. — DOI: 10.1109/ICCV.2017.324.
6. A Survey on Concept Drift Adaptation / J. Gama [и др.] // ACM Computing Surveys. — 2014. — Т. 46, № 4. — 44:1—44:37. — DOI: 10.1145/2523813.
7. INSOMNIA: Towards Concept-Drift Robustness in Network Intrusion Detection / G. Andresini [и др.] // 14th ACM Workshop on Artificial Intelligence and Security (AISec). — 2021. — С. 111—122. — DOI: 10.1145/3474369.3486864.
8. Moustafa N., Slay J. UNSW-NB15: A Comprehensive Data Set for Network Intrusion Detection Systems (UNSW-NB15 Network Data Set) // Military Communications and Information Systems Conference (MilCIS). — IEEE, 2015. — С. 1—6. — DOI: 10.1109/MilCIS.2015.7348942.

9. *Sharafaldin I., Lashkari A. H., Ghorbani A. A.* Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization // 4th International Conference on Information Systems Security and Privacy (ICISSP). — 2018. — C. 108—116. — DOI: 10.5220/0006639801080116.
10. Deep Learning for Cyber Security Intrusion Detection: Approaches, Datasets, and Comparative Study / M. A. Ferrag, L. Maglaras, S. Moschoyiannis, H. Janicke // Journal of Information Security and Applications. — 2020. — T. 50. — C. 102419. — DOI: 10.1016/j.jisa.2019.102419.
11. *Claise B.* Cisco Systems NetFlow Services Export Version 9. — IETF, 2004. — DOI: 10.17487/RFC3954.
12. *Claise B., Trammell B., Aitken P.* Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information. — IETF, 2013. — DOI: 10.17487/RFC7011.
13. *Liu F. T., Ting K. M., Zhou Z.-H.* Isolation Forest // IEEE International Conference on Data Mining (ICDM). — 2008. — C. 413—422. — DOI: 10.1109/ICDM.2008.17.
14. Estimating the Support of a High-Dimensional Distribution / B. Schölkopf [и др.] // Neural Computation. — 2001. — T. 13, № 7. — C. 1443—1471. — DOI: 10.1162/089976601750264965.
15. *Hochreiter S., Schmidhuber J.* Long Short-Term Memory // Neural Computation. — 1997. — T. 9, № 8. — C. 1735—1780. — DOI: 10.1162/neco.1997.9.8.1735.
16. Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation / K. Cho, B. van Merriënboer, C. Gulcehre [и др.] // Conference on Empirical Methods in Natural Language Processing (EMNLP). — 2014. — C. 1724—1734. — DOI: 10.3115/v1/D14-1179.
17. *Bai S., Kolter J. Z., Koltun V.* An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling. — 2018. — arXiv: 1803.01271 [cs.LG].
18. Attention Is All You Need / A. Vaswani, N. Shazeer, N. Parmar [и др.] // Advances in Neural Information Processing Systems (NeurIPS). T. 30. — 2017.

19. *Kingma D. P., Welling M.* Auto-Encoding Variational Bayes // International Conference on Learning Representations (ICLR). — 2014.
20. The 1999 DARPA Off-line Intrusion Detection Evaluation / R. P. Lippmann, D. J. Fried, I. Graf [и др.] // Computer Networks. — 2000. — Т. 34, № 4. — С. 579—595. — DOI: 10.1016/S1389-1286(00)00139-0.
21. An Evaluation Framework for Intrusion Detection Dataset / A. Gharib, I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani // International Conference on Information Science and Security (ICISA). — IEEE, 2016. — С. 1—6. — DOI: 10.1109/ICISSEC.2016.7885840.
22. Flow-based Network Traffic Generation Using Generative Adversarial Networks / M. Ring, D. Schlör, D. Landes, A. Hotho // Computers & Security. — 2019. — Т. 82. — С. 156—172. — DOI: 10.1016/j.cose.2018.12.012.
23. Conditional Variational Autoencoder for Prediction and Feature Recovery Applied to Intrusion Detection in IoT / M. Lopez-Martin, B. Carro, A. Sanchez-Esguevillas, J. Lloret // Sensors. — 2017. — Т. 17, № 9. — С. 1967. — DOI: 10.3390/s17091967.
24. Deep Reinforcement Adversarial Learning Against Network Intrusion Detection in Continuous-Time / G. Apruzzese [и др.] // IEEE Transactions on Network and Service Management. — 2020. — Т. 17, № 4. — С. 1975—1987. — DOI: 10.1109/TNSM.2020.3031843.
25. Generative Adversarial Nets / I. Goodfellow, J. Pouget-Abadie, M. Mirza [и др.] // Advances in Neural Information Processing Systems (NeurIPS). Т. 27. — 2014.
26. *Mirza M., Osindero S.* Conditional Generative Adversarial Nets. — 2014. — arXiv: 1411.1784 [cs.LG].
27. *Arjovsky M., Chintala S., Bottou L.* Wasserstein Generative Adversarial Networks // International Conference on Machine Learning (ICML). — 2017. — С. 214—223.
28. Improved Training of Wasserstein GANs / I. Gulrajani [и др.] // Advances in Neural Information Processing Systems (NeurIPS). Т. 30. — 2017.

29. A Detailed Analysis of the KDD CUP 99 Data Set / M. Tavallaei, E. Bagheri, W. Lu, A. A. Ghorbani // IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA). — IEEE, 2009. — C. 1—6. — DOI: 10.1109/CISDA.2009.5356528.
30. *Moustafa N.* A New Distributed Architecture for Evaluating AI-based Security Systems at the Edge: Network TON_IoT Datasets // Sustainable Cities and Society. — 2021. — T. 72. — C. 102994. — DOI: 10.1016/j.scs.2021.102994.
31. Towards the Development of Realistic Botnet Dataset in the Internet of Things for Network Forensic Analytics: Bot-IoT Dataset / N. Koroniotis, N. Moustafa, E. Sitnikova, B. Turnbull // Future Generation Computer Systems. — 2019. — T. 100. — C. 779—796. — DOI: 10.1016/j.future.2019.05.041.
32. Statistical Analysis of Honeypot Data and Building of Kyoto 2006+ Dataset for NIDS Evaluation / J. Song [и др.] // 1st Workshop on Building Analysis Datasets and Gathering Experience Returns for Security (BADGERS). — 2011. — C. 29—36. — DOI: 10.1145/1978672.1978676.
33. LITNET-2020: An Annotated Real-World Network Flow Dataset for Network Intrusion Detection / R. Damasevicius, A. Venckauskas, S. Grigaliunas [и др.] // Electronics. — 2020. — T. 9, № 5. — C. 800. — DOI: 10.3390/electronics9050800.
34. On Calibration of Modern Neural Networks / C. Guo, G. Pleiss, Y. Sun, K. Q. Weinberger // International Conference on Machine Learning (ICML). — 2017. — C. 1321—1330.
35. TESSERACT: Eliminating Experimental Bias in Malware Classification across Space and Time / F. Pendlebury [и др.] // USENIX Security Symposium. — 2019. — C. 729—746.
36. Long Short Term Memory Recurrent Neural Network Classifier for Intrusion Detection / J. Kim, J. Kim, H. L. T. Thu, H. Kim // International Conference on Platform Technology and Service (PlatCon). — IEEE, 2016. — C. 1—5. — DOI: 10.1109/PlatCon.2016.7456805.

37. A Deep Learning Approach for Intrusion Detection Using Recurrent Neural Networks / C. Yin, Y. Zhu, J. Fei, X. He // IEEE Access. — 2017. — T. 5. — C. 21954—21961. — DOI: 10.1109/ACCESS.2017.2762418.
38. *Moustafa N., Slay J.* The Evaluation of Network Anomaly Detection Systems: Statistical Analysis of the UNSW-NB15 Data Set and the Comparison with the KDD99 Data Set // Information Security Journal: A Global Perspective. — 2016. — T. 25, № 1—3. — C. 18—31. — DOI: 10.1080/19393555.2015.1125974.
39. Deep Learning Approach for Intelligent Intrusion Detection System / R. Vinayakumar [и др.] // IEEE Access. — 2019. — T. 7. — C. 41525—41550. — DOI: 10.1109/ACCESS.2019.2895334.
40. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection / Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai // Network and Distributed System Security Symposium (NDSS). — 2018. — DOI: 10.14722/ndss.2018.23204.
41. *Liu H., Lang B.* Machine Learning and Deep Learning Methods for Intrusion Detection Systems: A Survey // Applied Sciences. — 2019. — T. 9, № 20. — C. 4396. — DOI: 10.3390/app9204396.
42. *The UNSW Canberra Cyber.* The UNSW-NB15 Dataset Description. — 2015. — <https://research.unsw.edu.au/projects/unsw-nb15-dataset>.

ПРИЛОЖЕНИЯ

Приложение А. Полная таблица результатов эксперимента E1

В таблица А.1 приведена полная сводка результатов 22 запусков эксперимента E1. Для каждой комбинации (модель, seed) зафиксированы: оптимальный порог τ (под target FPR = 0,01 на val), температура калибровки, ECE до/после калибровки, Precision, Recall, F1, FPR, MCC, ROC-AUC, PR-AUC с bootstrap-CI 95 %, latency на одно окно.

Таблица А.1 – Полная сводка результатов эксперимента E1 (22 запуска)

Модель	seed	τ	F1	PR-AUC	Prec.	Rec.	FPR	Lat., mc
logistic_regression	42	0.502	0.847	0.986	0.991	0.739	0.014	0.11
logistic_regression	123	0.502	0.847	0.986	0.991	0.739	0.014	0.12
logistic_regression	2024	0.502	0.847	0.986	0.991	0.739	0.014	0.08
random_forest	42	0.512	0.917	0.993	0.992	0.853	0.014	54.83
random_forest	123	0.515	0.917	0.993	0.991	0.852	0.016	52.74
random_forest	2024	0.510	0.917	0.993	0.993	0.852	0.013	46.95
xgboost	42	0.498	0.919	0.993	0.990	0.857	0.019	0.46
xgboost	123	0.498	0.919	0.993	0.990	0.857	0.019	0.74
xgboost	2024	0.498	0.919	0.993	0.990	0.857	0.019	0.87
isolation_forest	42	0.952	0.121	0.872	0.988	0.064	0.001	21.02
isolation_forest	123	0.910	0.453	0.879	0.985	0.296	0.009	21.34
isolation_forest	2024	0.940	0.198	0.872	0.987	0.110	0.004	21.57
cnn_lstm	42	0.675	0.845	0.910	0.992	0.561	0.033	0.95
cnn_lstm	123	0.682	0.722	0.975	0.987	0.569	0.013	1.12
cnn_lstm	2024	0.680	0.743	0.980	0.987	0.595	0.012	0.82
mlp	42	0.500	0.809	0.715	0.679	1.000	1.000	0.18
cnn1d	42	0.700	0.674	0.979	0.993	0.510	0.008	0.69
lstm	42	0.690	0.821	0.985	0.991	0.701	0.013	0.82
gru	42	0.685	0.821	0.985	0.988	0.703	0.018	3.39
bilstm	42	0.695	0.842	0.988	0.992	0.731	0.012	1.84
tcn	42	0.692	0.844	0.986	0.992	0.735	0.013	4.20
transformer	42	0.700	0.773	0.981	0.989	0.635	0.016	1.44

Сырая таблица в формате CSV (E1_results.csv) поставляется в составе артефактов программного прототипа.

Приложение Б. Структура программного прототипа

Прототип реализован как Python-пакет `diploma_nids` с подпакетами `data`, `models`, `training`, `eval`, `attacker`, `inference`, `ui`, `utils`. Полное описание модулей и тестов приведено на этапе 3 настоящего отчёта.

Запуск экспериментов (из корневой директории программного прототипа):

```
make install-dev
python scripts/02_build_dataset.py \
    --config configs/data/unswnb15.yaml \
    --preprocess configs/preprocess/default.yaml \
    --strategy official
python scripts/10_run_experiment_E1.py \
    --train configs/train/full.yaml --seeds 42 123 2024
python scripts/11_run_experiment_E2_error_analysis.py
python scripts/12_run_experiment_E4_attacker.py
python scripts/13_run_experiment_E5_drift.py
python scripts/20_build_thesis_tables.py
python scripts/22_parse_training_logs.py
python scripts/21_plot_training_history.py
python scripts/30_demo_run.py
```

Приложение В. Конфигурации экспериментов

Все стадии pipeline параметризуются YAML-файлами в каталоге `configs/` программного прототипа. Ключевые группы конфигов:

- `configs/data/` — схемы UNSW-NB15 и CICIDS2017 (mapping общих признаков для cross-evaluation);
- `configs/preprocess/default.yaml` — параметры очистки, encoding, log/robust scaling, windowing, splits, балансировки;
- `configs/models/` — архитектуры 10 DL-моделей (`cnn_lstm.yaml` — proposed) и параметры 5 классических моделей (`classical.yaml`);
- `configs/train/` — параметры обучения (`full.yaml` для полного режима, `smoke.yaml` для smoke-тестов);

- `configs/eval/default.yaml` — метрики, калибровка, bootstrap CI, drift-monitoring;
- `configs/attacker/` — политика FSM (`policy.yaml`), 7 шаблонов атак, drift-инжектор (`drift.yaml`);
- `configs/pipeline/realtime.yaml` — конфигурация realtime-стенда FastAPI + Streamlit + agent.

Каждый запуск экспериментов фиксирует: версию кода (commit hash), набор seed, гиперпараметры, версии зависимостей, полное содержимое использованных конфигов в виде attached-артефактов.